ENGENHARIA DE SOFTWARE I — TRABALHO II

Veridit

Documento de Arquitetura de Software

Professor: Dr. Eduardo Almeida

Equipe: <nomes da equipe>

Base: Documento de Requisitos Veridit v01.00 (REQ01–REQ15, D1, D2, Apêndice A)

Referência: Richards & Ford, *Fundamentals of Software Architecture*

Versão: 1.0 — 15/10/2026 · Universidade Federal da Bahia

Sumário

1. 1. Introdução
2. 2. Características arquiteturais
3. 3. Top 4 características arquiteturais
4. 4. Estilo arquitetural
5. 5. Componentes candidatos
6. 6. Diagrama de componentes
7. 7. Especificação interna dos componentes críticos
8. 8. Diagrama de módulos (pacotes UML)
9. 9. Táticas por característica
10. 10. Decisões arquiteturais (ADRs)
11. 11. Rastreabilidade: requisitos → componentes
12. 12. Riscos e pontos em aberto
13. 13. Telas da aplicação
14. Referências
15. Apêndice A — Registros de Decisão Arquitetural (ADRs)

1. Introdução

O Veridit é uma plataforma para registrar conteúdo da internet. O usuário (advogado ou usuário comum) compra créditos, informa o endereço de um site e captura uma tela ou grava um vídeo da navegação. Ao final, recebe um relatório e um arquivo .zip com o que foi capturado, que podem ser usados como prova.

Este documento descreve a arquitetura proposta seguindo a abordagem de Richards & Ford (*Fundamentals of Software Architecture*): identificação das características arquiteturais, escolha das prioritárias, identificação de componentes, escolha do estilo e registro das decisões em ADRs. As táticas de cada característica seguem o catálogo de Bass, Clements & Kazman (*Software Architecture in Practice*).

1.1 Premissas e interpretações dos requisitos

#	Premissa	Validar com o cliente?
P1	O documento de requisitos tem dois "REQ 11". Chamamos o segundo ("Confirmar captura de tela por email") de **REQ11-B**.	Sim
P2	"Pagamento pela plataforma do Veridit" (REQ06) significa pagar via Pix sem sair da interface do Veridit; o QR Code é gerado pelo provedor de pagamento (ADR-0006).	Sim
P3	A quantidade de créditos consumida por registro (tela ou vídeo, duração do vídeo) não está definida; será configurável no catálogo (ADR-0008).	Sim
P4	O registro tende a ser usado como prova, já que advogados são público-alvo (D1). Isso torna a integridade das evidências uma característica implícita.	Sim
P5	Nesta etapa não há implementação; tecnologias citadas (PostgreSQL, RabbitMQ, Playwright etc.) são exemplos que podem ser trocados por equivalentes.	Não

2. Características arquiteturais

Identificamos as características a partir de três fontes, como propõem Richards & Ford: os requisitos explícitos (o atributo de qualidade "Segurança"), os requisitos funcionais que implicam características (pagamento, e-mails obrigatórios, captura de vídeo) e o conhecimento do domínio (registro de conteúdo com valor de prova). Classificamos cada uma como operacional, estrutural ou transversal e indicamos se é explícita ou implícita.

O livro alerta contra a "arquitetura genérica", que tenta suportar todas as características e acaba complexa sem atender bem a nenhuma. Por isso, listamos abaixo todas as características relevantes, mas só quatro direcionam a estrutura (seção 3); as demais são atendidas por táticas locais (seção 9).

#	Característica	Categoria	Origem	Justificativa	Requisitos
1	**Segurança**	Transversal	Explícita	Único atributo de qualidade do documento: acesso só por usuários cadastrados. O sistema guarda CPF, OAB, endereço e lida com pagamento.	QA Segurança, REQ01–REQ07
2	**Integridade e autenticidade das evidências**	Transversal	Implícita	O relatório e o .zip só têm valor se for possível provar que não foram alterados e quando foram obtidos.	REQ08–REQ15, D1
3	**Confiabilidade**	Operacional	Implícita	O usuário paga antes de usar: pagamento confirmado precisa virar crédito, e crédito não pode ser perdido quando a captura falha. Os e-mails de confirmação são de prioridade alta.	REQ05–REQ11-B
4	**Escalabilidade / elasticidade**	Operacional	Implícita	Cada captura ocupa um navegador dedicado por minutos (CPU, memória, vídeo), com demanda irregular.	REQ08–REQ11
5	Disponibilidade	Operacional	Implícita	Um conteúdo pode sair do ar a qualquer momento; se o sistema estiver fora, a prova se perde.	REQ03, REQ08
6	Desempenho	Operacional	Implícita	A navegação no navegador remoto precisa ser fluida; listagens e detalhes precisam responder rápido.	REQ08, REQ12, REQ13
7	Recuperabilidade	Operacional	Implícita	Evidências e o livro-razão de créditos não podem ser perdidos em falhas graves.	REQ12–REQ15
8	Auditabilidade	Transversal	Implícita	Cadeia de custódia das evidências e histórico de créditos para contestações.	REQ05, REQ08–REQ15
9	Privacidade (LGPD)	Transversal	Implícita	Dados pessoais (CPF, telefone, endereço, OAB) e dados de terceiros nas capturas.	Apêndice A, REQ01, REQ05
10	Extensibilidade / configurabilidade	Estrutural	Implícita	Empresas como novo tipo de usuário (D1), pacotes definidos pelo negócio (D2), novos tipos de captura.	D1, D2, REQ05
11	Interoperabilidade	Estrutural	Implícita	Integração com Mercado Pago, Pix, provedor de e-mail e Autoridade de Carimbo do Tempo.	REQ06, REQ07, REQ10, REQ11-B
12	Manutenibilidade e testabilidade	Estrutural	Implícita	O projeto terá etapas de implementação e evolução feitas pela equipe.	Todos

3. Top 4 características arquiteturais

> Decisão registrada em ADR-0001.

Prioridade	Característica	Por que está no top 4	Cenário mensurável (meta inicial)
1	Segurança	Explícita; dados pessoais e de pagamento	100% dos endpoints não públicos rejeitam requisição sem token válido; nenhuma senha armazenada de forma reversível
2	Integridade e autenticidade das evidências	Sem ela o produto perde o valor de prova	Alterar qualquer byte do pacote é detectado pela verificação; 100% dos registros concluídos têm hash e carimbo do tempo
3	Confiabilidade	Dinheiro e créditos do usuário	Com webhooks duplicados ou queda de serviço no meio do fluxo, 0 créditos duplicados ou perdidos
4	Escalabilidade (elasticidade da captura)	A captura tem perfil de carga diferente de todo o resto	10x sessões simultâneas absorvidas só adicionando workers; espera p95 por sessão ≤ 30 s

Escolhemos as características cuja falha torna o produto inútil ou perigoso e que mudam a forma da arquitetura. Segurança é a única explícita. Integridade é o núcleo do valor: uma evidência contestável não serve ao advogado. Confiabilidade vem do modelo pré-pago de créditos. Escalabilidade vem do custo e da irregularidade da captura. Juntas, essas quatro forças levam a separar a captura do núcleo transacional e a usar comunicação assíncrona.

Ficaram fora do top 4: disponibilidade (sem SLA definido; já beneficiada pela redundância), desempenho (limitado pelo site de destino), auditabilidade e privacidade (cobertas por táticas de integridade e segurança) e extensibilidade (atendida no nível dos componentes). A ADR-0001 detalha as alternativas.

4. Estilo arquitetural

> Decisão registrada em ADR-0002.

O Veridit adota o estilo **baseado em serviços** (*service-based*), particionado por domínio, com **mensageria assíncrona** onde a assincronia é natural (captura, selagem, e-mails e auditoria).

Comparação das notas de Richards & Ford para as características que dependem do estilo:

Característica	Camadas	Microkernel	**Baseado em serviços**	Orientado a eventos	Space-based	Microsserviços
Escalabilidade	1	1	**3**	5	5	5
Elasticidade	1	1	**2**	3	5	5
Tolerância a falhas	1	1	**4**	5	3	4
Confiabilidade	3	3	**4**	3	4	4
Testabilidade	2	3	**4**	2	1	4
Simplicidade	5	4	**3**	1	1	1
Custo (5 = barato)	5	5	**4**	3	2	1

O estilo baseado em serviços tem boas notas em tolerância a falhas, confiabilidade, testabilidade e custo, com simplicidade razoável para a equipe. Sua fraqueza (elasticidade 2) é tratada isolando a captura em um serviço com workers sem estado alimentados por fila (ADR-0009). A separação em serviços também isola o navegador, que abre sites externos não confiáveis, dos dados pessoais e de pagamento.

4.1 Serviços e implantação

Serviço	Componentes	Escala	Dados	Comunicação
Borda	C01, C02	2+ réplicas	—	HTTPS, WebSocket
S1 Identidade e Acesso	C03, C04, C05	2+ réplicas	esquema `identidade`	REST; publica eventos
S2 Créditos e Pagamentos	C06, C07, C08, C09	2+ réplicas	esquema `creditos`	REST; webhooks; publica eventos
S3 Registros	C10, C11	2+ réplicas	esquema `registros`	REST; consome e publica eventos
S4 Captura	C12	**elástico** (mín. 2, máx. N)	sem banco; bucket temporário	fila + WebSocket
S5 Evidências	C13, C14, C15, C16	2+ réplicas	esquema `evidencias` ; bucket WORM	eventos; REST de download
S6 Notificações	C17	1–2 réplicas	sem banco	eventos → e-mail
S7 Auditoria	C18	1–2 réplicas	esquema `auditoria`	eventos

Quanta arquiteturais. Os serviços que compartilham o banco (S1, S2, S3, S5, S7) formam um quantum. A Captura (S4) e as Notificações (S6) não acessam o banco e se comunicam só de forma assíncrona, por isso formam quanta próprios. Isso permite que a Captura tenha características operacionais diferentes (elasticidade) sem afetar o restante.

5. Componentes candidatos

Identificamos os componentes com a abordagem **ator/ações** e validamos com a abordagem de **fluxo de trabalho** (compra de créditos e registro de conteúdo), evitando a "armadilha da entidade" do livro: componentes são nomeados pelo que fazem (ex.: *Orquestrador de Captura*), não pelas tabelas que manipulam (ex.: *RegistroManager*).

5.1 Atores e ações

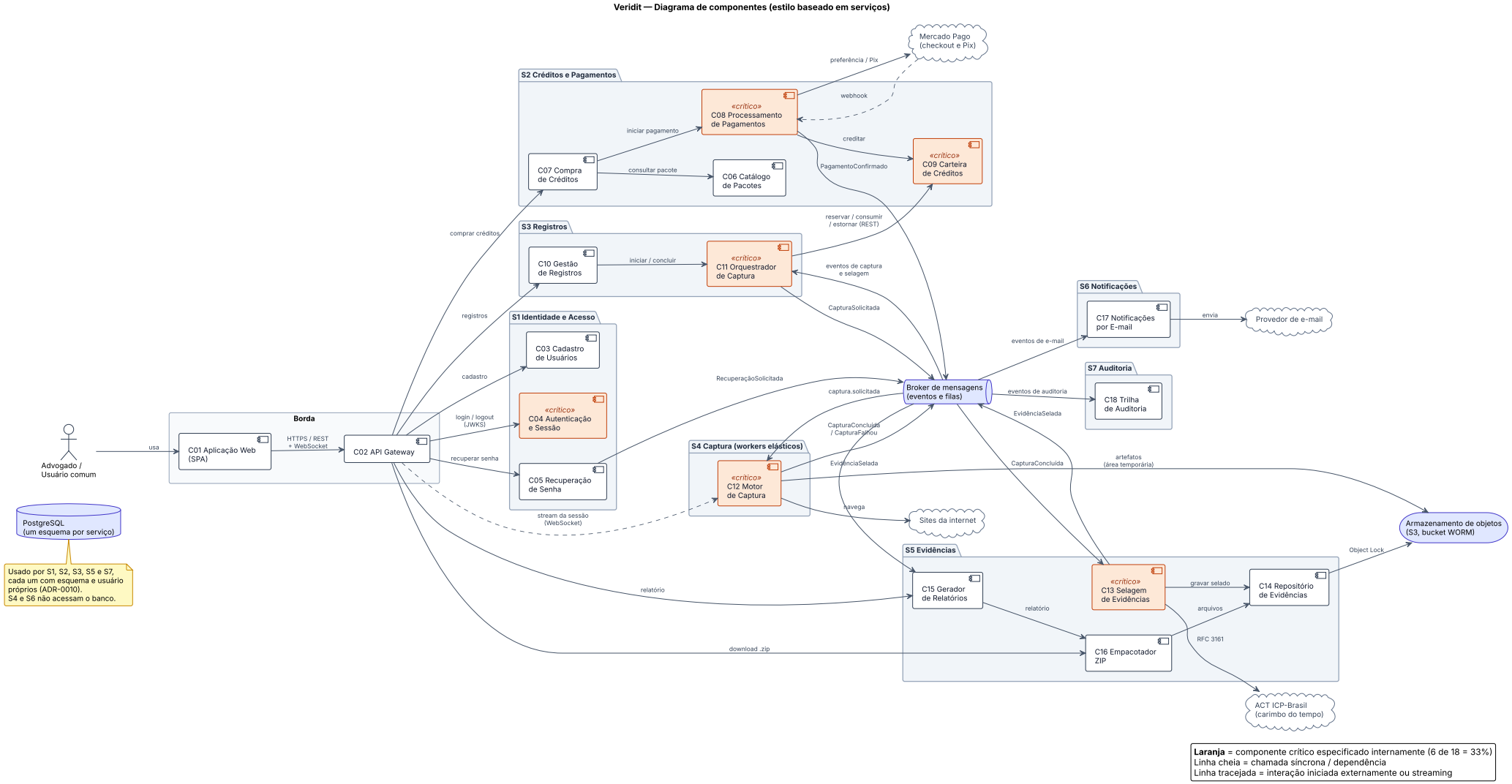
Ator	Ações
Advogado / Usuário comum	cadastrar-se, recuperar senha, entrar, sair, comprar créditos, pagar, iniciar registro, navegar e capturar, concluir, listar registros, ver detalhes, baixar relatório e .zip
Mercado Pago (sistema externo)	notificar pagamento (webhook)
Sistema (tempo)	expirar reservas de créditos, reconciliar pagamentos, verificar integridade das evidências, escalar workers
Empresa (futuro, D1)	as mesmas ações do usuário, com dados de pessoa jurídica

5.2 Lista de componentes

ID	Componente	Serviço	Responsabilidades	Requisitos	Características
C01	Aplicação Web (SPA)	Borda	Telas de cadastro, login, compra, sessão de captura (visualização do navegador remoto), listagem, detalhes e downloads	REQ01–REQ15	Segurança, Desempenho
C02	API Gateway	Borda	Ponto único de entrada: TLS, validação de JWT, lista de revogação, limitação de taxa, roteamento REST e proxy WebSocket das sessões	Todos	Segurança, Escalabilidade
C03	Cadastro de Usuários	S1	Cadastrar advogado (nome, e-mail, senha, CPF, OAB) e usuário comum (nome, e-mail, senha, CPF); validar CPF e OAB; verificar e-mail; preparado para empresa (D1)	REQ01, D1	Segurança, Privacidade, Extensibilidade
C04	Autenticação e Sessão	S1	Login, emissão e renovação de tokens, logout com revogação, proteção contra força bruta, publicação de chaves (JWKS)	REQ03, REQ04	Segurança
C05	Recuperação de Senha	S1	Gerar token de uso único, solicitar envio do link, redefinir senha e revogar sessões	REQ02	Segurança
C06	Catálogo de Pacotes	S2	Manter pacotes (nome, quantidade de créditos, valor por crédito, benefícios) e a política de consumo; pacotes básico, médio e premium	REQ05, D2	Configurabilidade
C07	Compra de Créditos	S2	Seleção do pacote, coleta e validação dos dados de faturamento (Apêndice A), criação do pedido	REQ05	Confiabilidade, Privacidade
C08	Processamento de Pagamentos	S2	Integração com Mercado Pago (checkout e Pix com QR Code/copia-e-cola), webhooks, confirmação idempotente, reconciliação	REQ06, REQ07	Segurança, Confiabilidade, Interoperabilidade
C09	Carteira de Créditos	S2	Livro-razão: creditar compras; reservar, consumir e estornar créditos de registros; saldo	REQ05, REQ08	Confiabilidade, Auditabilidade

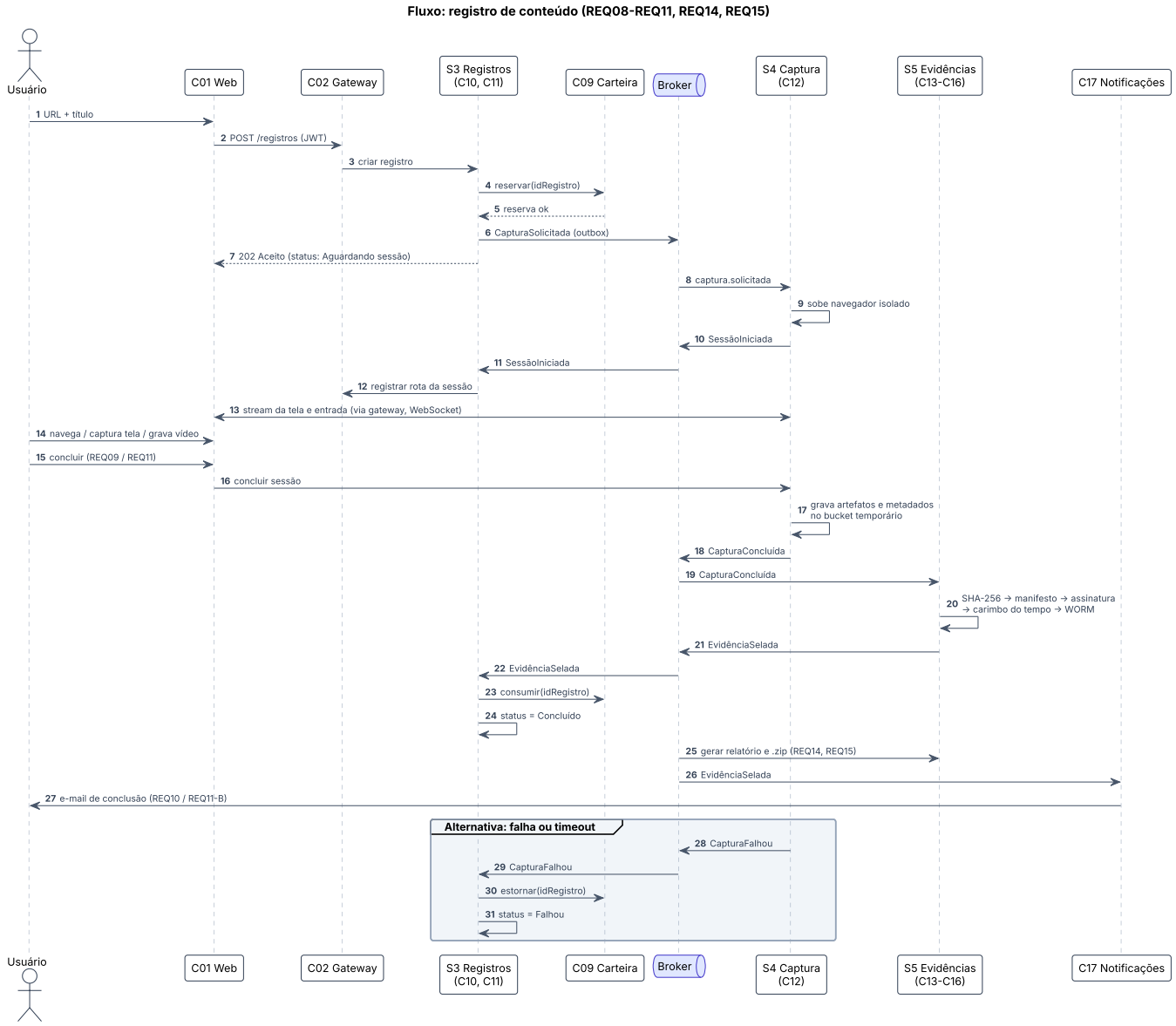
ID	Componente	Serviço	Responsabilidades	Requisitos	Características
C10	Gestão de Registros	S3	Criar registro (URL, título), listar (título, início, fim, detalhes, status), exibir detalhes e links de relatório e .zip	REQ08, REQ12, REQ13	Segurança, Desempenho
C11	Orquestrador de Captura	S3	Saga do registro: reserva de créditos, solicitação de sessão, prazos, conclusão, consumo ou estorno, atualização de status	REQ08–REQ11	Confiabilidade, Escalabilidade
C12	Motor de Captura	S4	Navegador remoto isolado, streaming, captura de tela, gravação de vídeo, coleta de metadados, publicação dos artefatos	REQ08, REQ09, REQ11	Integridade, Segurança, Escalabilidade
C13	Selagem de Evidências	S5	Hash SHA-256, manifesto, assinatura, carimbo do tempo, cadeia de custódia	REQ09, REQ11, REQ14	Integridade, Auditabilidade
C14	Repositório de Evidências	S5	Guardar arquivos selados em armazenamento imutável; metadados (tipos de dados, nº de imagens e vídeos); URLs de download temporárias	REQ13, REQ15	Integridade, Recuperabilidade
C15	Gerador de Relatórios	S5	Gerar relatório (PDF) com os campos do Apêndice A, hashes e instruções de verificação	REQ14	Integridade
C16	Empacotador ZIP	S5	Montar o .zip com arquivos, manifesto, assinatura, carimbo e relatório	REQ15	Integridade, Desempenho
C17	Notificações por E-mail	S6	Enviar e-mails a partir de eventos (pagamento, gravação concluída, captura de tela concluída, recuperação de senha, verificação de cadastro), com templates e retentativas	REQ02, REQ07, REQ10, REQ11-B	Confiabilidade
C18	Trilha de Auditoria	S7	Registrar eventos de segurança, créditos e custódia em log somente de inserção com encadeamento de hashes	QA Segurança	Auditabilidade, Integridade

6. Diagrama de componentes



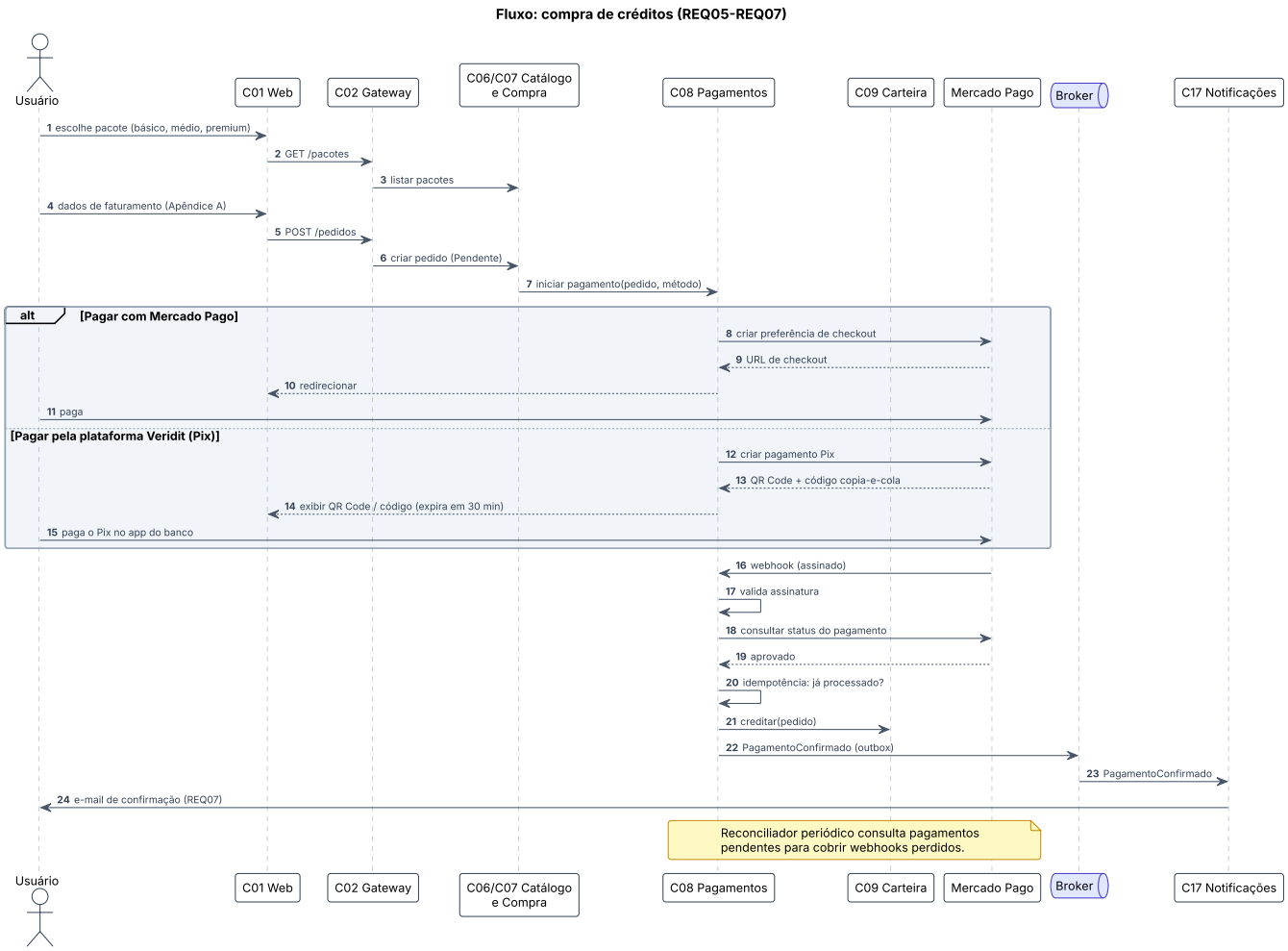
Os componentes em laranja são os críticos, especificados internamente na seção 7. As linhas cheias indicam chamadas e dependências; as tracejadas indicam interações iniciadas por fora (webhook) ou o streaming da sessão.

6.1 Fluxo principal: registro de conteúdo



Fluxo de registro

6.2 Fluxo principal: compra de créditos



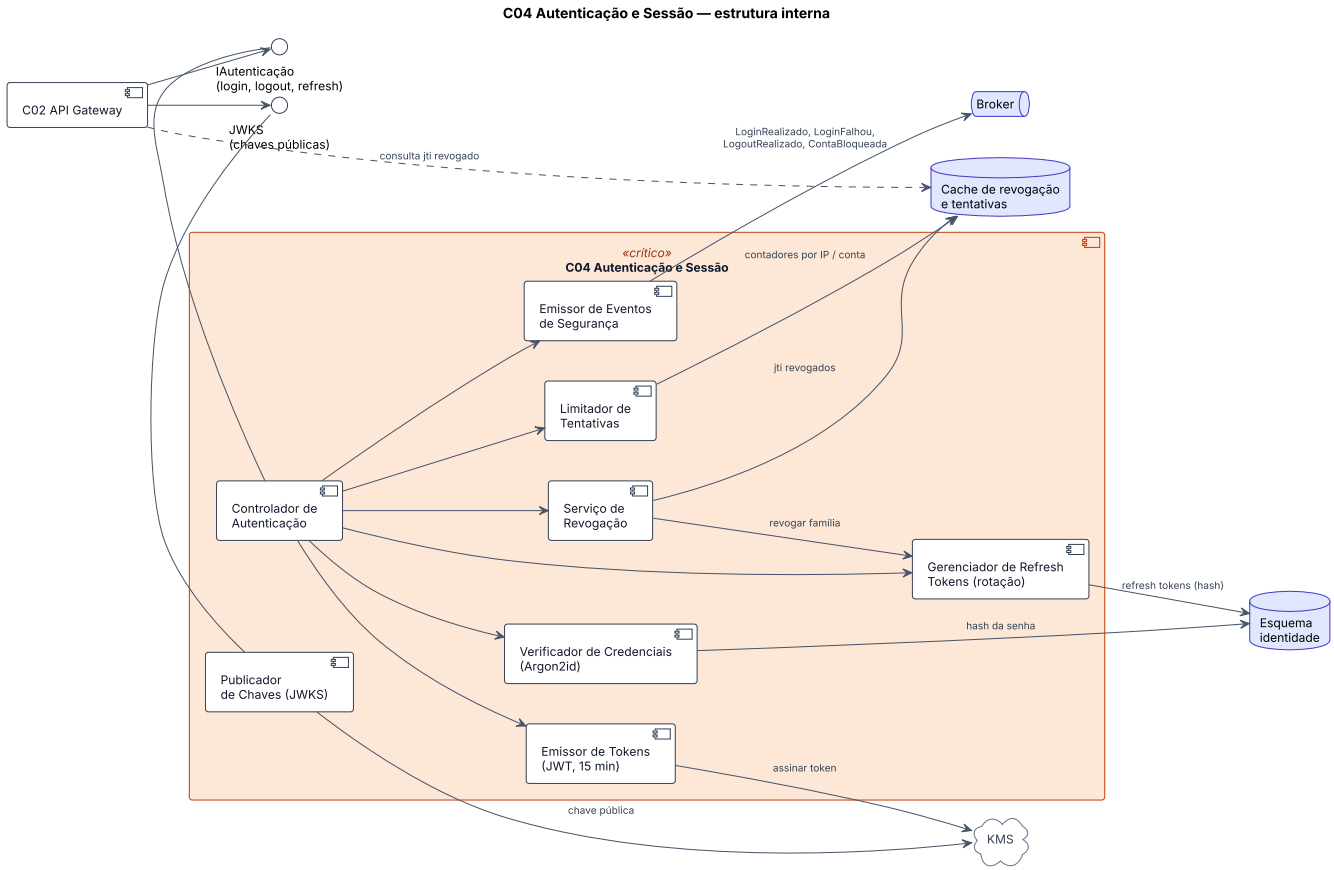
Fluxo de compra

7. Especificação interna dos componentes críticos

Especificamos internamente **6 dos 18 componentes (33%)**. O critério foi: o componente é essencial para pelo menos uma das top 4 características e sua falha compromete o produto. Cada característica prioritária é coberta por pelo menos um componente detalhado.

Componente	Por que é crítico	Características
C04 Autenticação e Sessão	Porta de entrada de todo acesso; uma falha compromete o único atributo explícito	Segurança
C08 Processamento de Pagamentos	Lida com dinheiro e com integração externa assíncrona (webhooks)	Segurança, Confiabilidade
C09 Carteira de Créditos	Guarda o saldo do usuário; corridas ou perdas geram prejuízo direto	Confiabilidade
C11 Orquestrador de Captura	Coordena o fluxo distribuído mais longo e mantém créditos e status coerentes	Confiabilidade, Escalabilidade
C12 Motor de Captura	Origem da evidência; executa conteúdo não confiável; é o mais caro e o que precisa escalar	Integridade, Segurança, Escalabilidade
C13 Selagem de Evidências	Transforma arquivos em evidência verificável; sem ele o produto perde o valor	Integridade

7.1 C04 Autenticação e Sessão



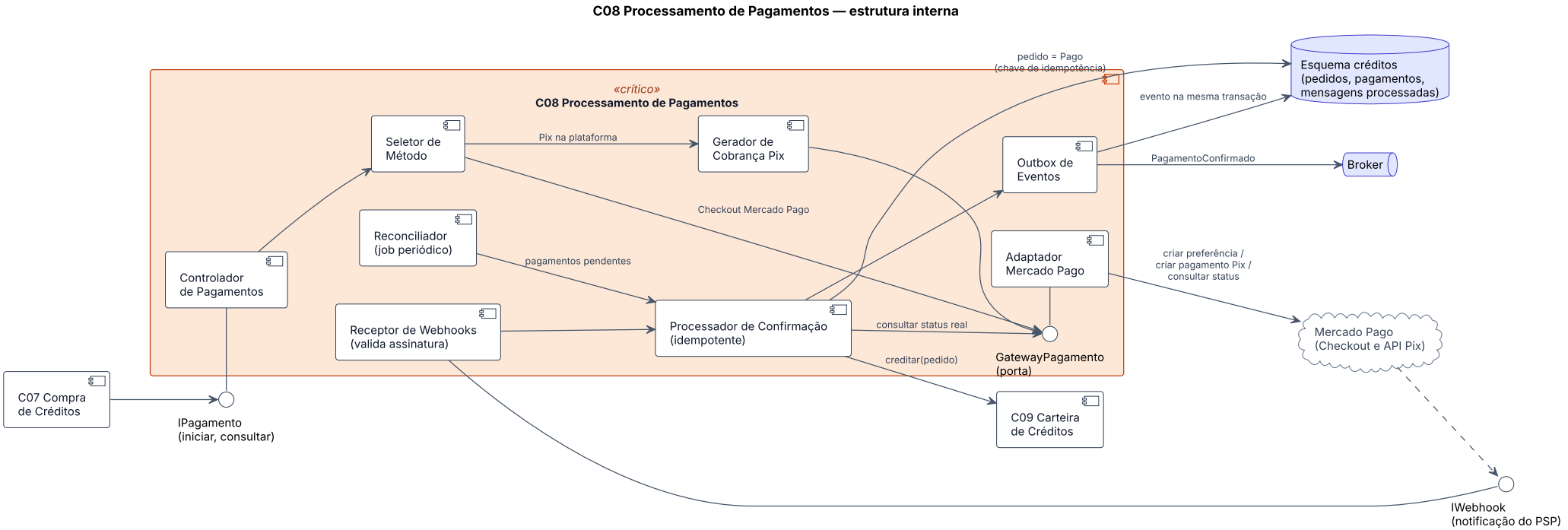
C04

Subcomponente	Responsabilidade
Controlador de Autenticação	Recebe login, refresh e logout; coordena os demais

Subcomponente	Responsabilidade
Limitador de Tentativas	Conta falhas por IP e por conta; aplica bloqueio progressivo
Verificador de Credenciais	Compara a senha com o hash Argon2id
Emissor de Tokens	Gera o JWT de acesso (15 min), assinado via KMS
Gerenciador de Refresh Tokens	Emite, rotaciona e detecta reuso de refresh tokens
Serviço de Revogação	Revoga tokens no logout, na troca de senha e em reuso suspeito
Publicador de Chaves (JWKS)	Expõe as chaves públicas para o gateway validar tokens
Emissor de Eventos de Segurança	Publica eventos para a trilha de auditoria

Interfaces: provê `IAutenticação` e `JWKS`; requer KMS, esquema `identidade`, cache de revogação e broker. Decisão relacionada: ADR-0005.

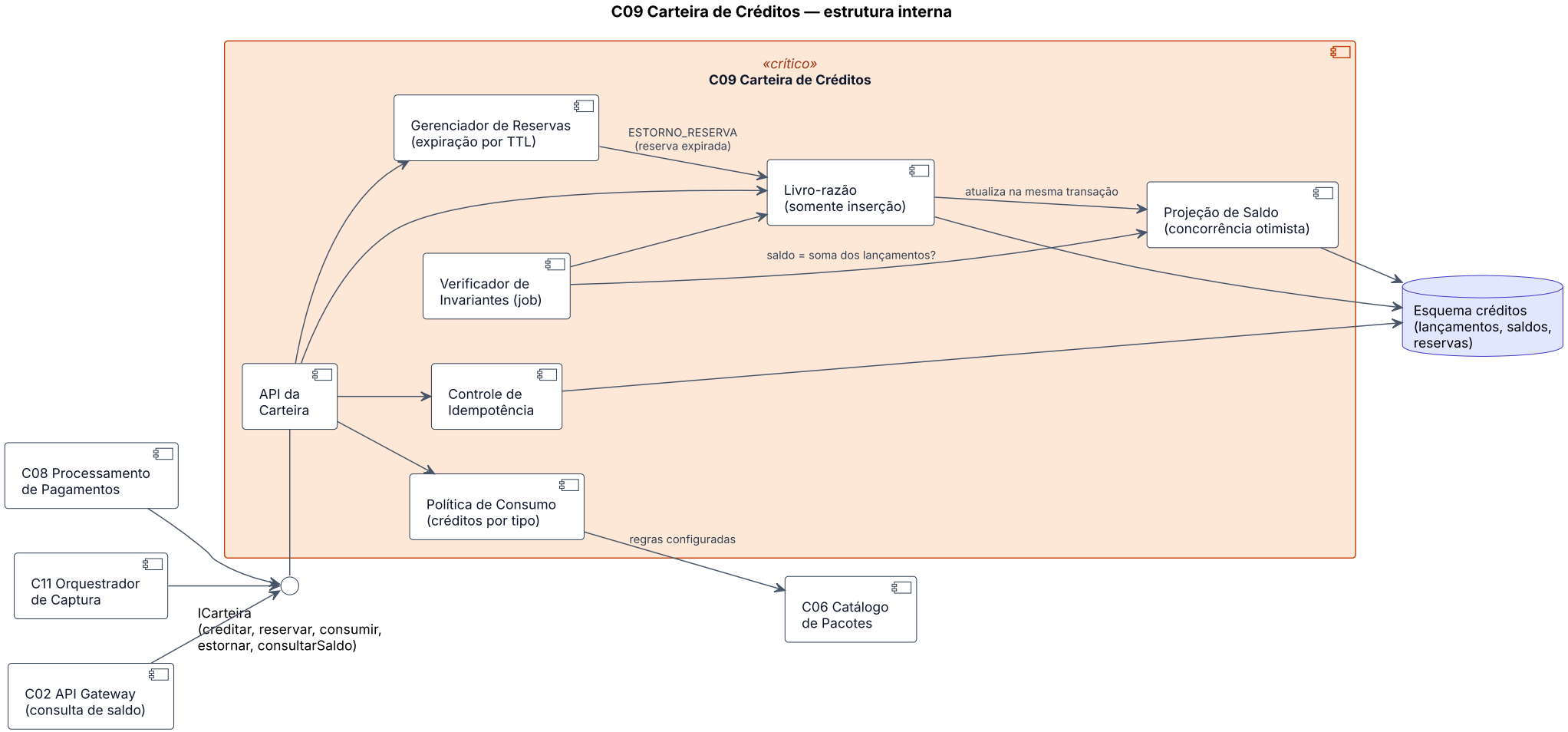
7.2 C08 Processamento de Pagamentos



Subcomponente	Responsabilidade
Controlador de Pagamentos	Recebe o pedido de pagamento da Compra de Créditos
Seletor de Método	Decide entre checkout do Mercado Pago e Pix na plataforma
Gerador de Cobrança Pix	Obtém QR Code e código copia-e-cola, com expiração
Porta `GatewayPagamento` + Adaptador Mercado Pago	Isola o modelo do provedor (camada anticorrupção); permite trocar de provedor
Receptor de Webhooks	Recebe notificações e valida a assinatura
Processador de Confirmação	Consulta o status real, aplica idempotência, marca o pedido como pago e credita a carteira
Reconciliador	Job periódico que confere pagamentos pendentes para cobrir webhooks perdidos
Outbox de Eventos	Grava `PagamentoConfirmado` na mesma transação e publica no broker

Decisões relacionadas: ADR-0006, ADR-0007.

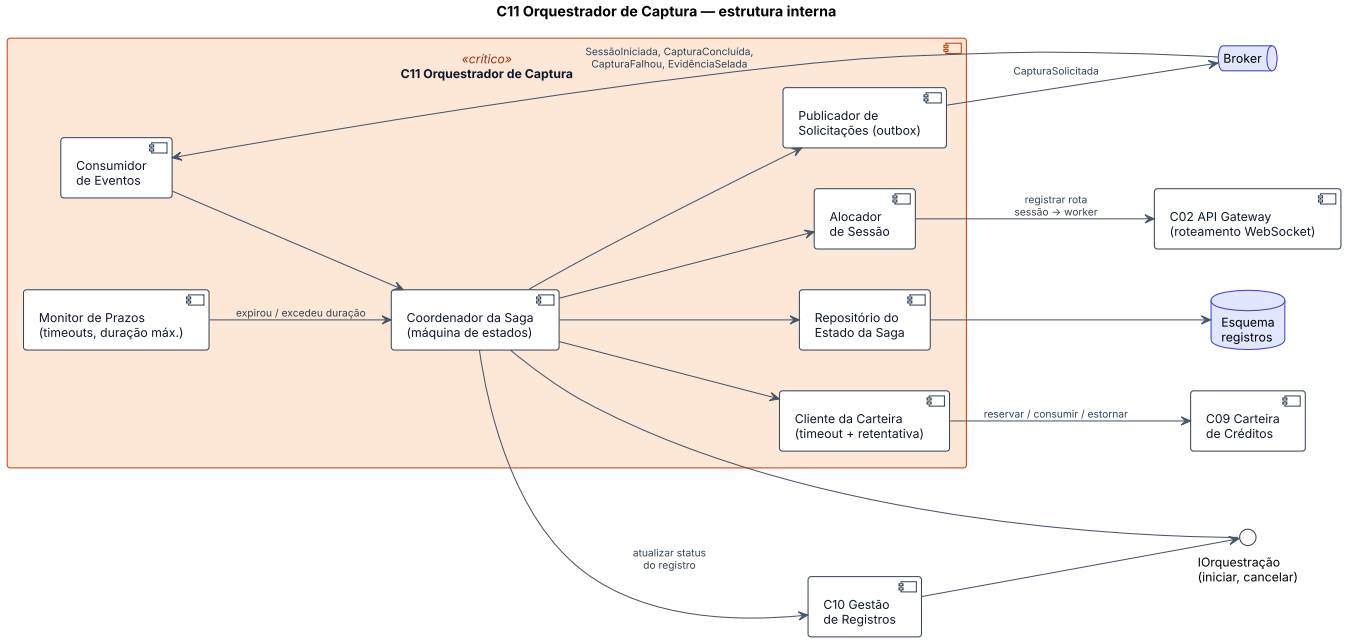
7.3 C09 Carteira de Créditos



Subcomponente	Responsabilidade
API da Carteira	Operações `creditar`, `reservar`, `consumir`, `estornar`, `consultarSaldo`
Controle de Idempotência	Garante que cada operação com a mesma chave tenha efeito uma vez
Política de Consumo	Calcula quantos créditos um registro consome, a partir do catálogo
Livro-razão	Lançamentos somente de inserção (compra, reserva, consumo, estorno, ajuste)
Projeção de Saldo	Saldo materializado com concorrência otimista, atualizado na mesma transação
Gerenciador de Reservas	Estorna reservas expiradas
Verificador de Invariantes	Job que confere se o saldo é igual à soma dos lançamentos

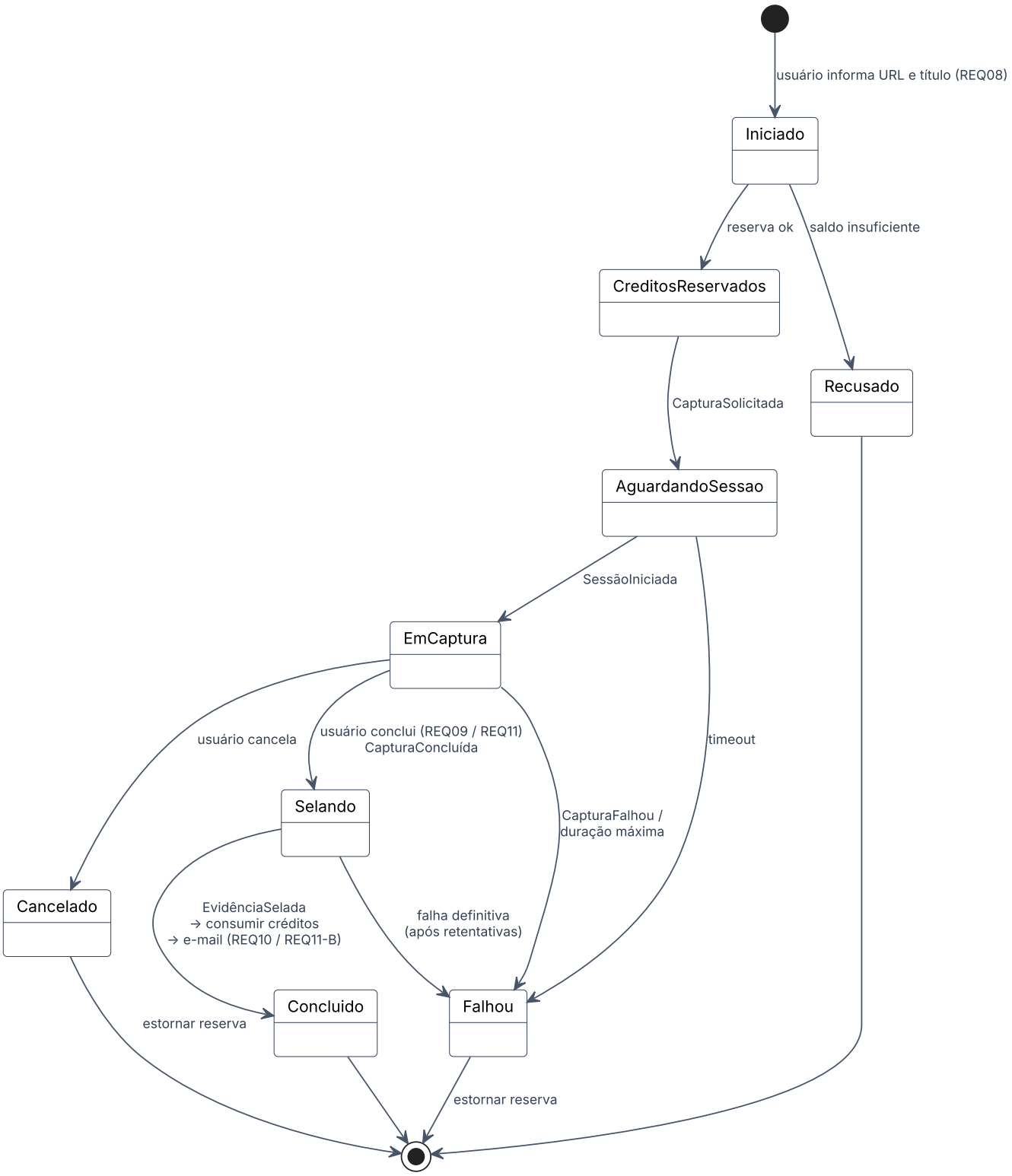
Decisão relacionada: ADR-0008.

7.4 C11 Orquestrador de Captura



C11

Estados do registro controlados pela saga (C11)



Estados do registro

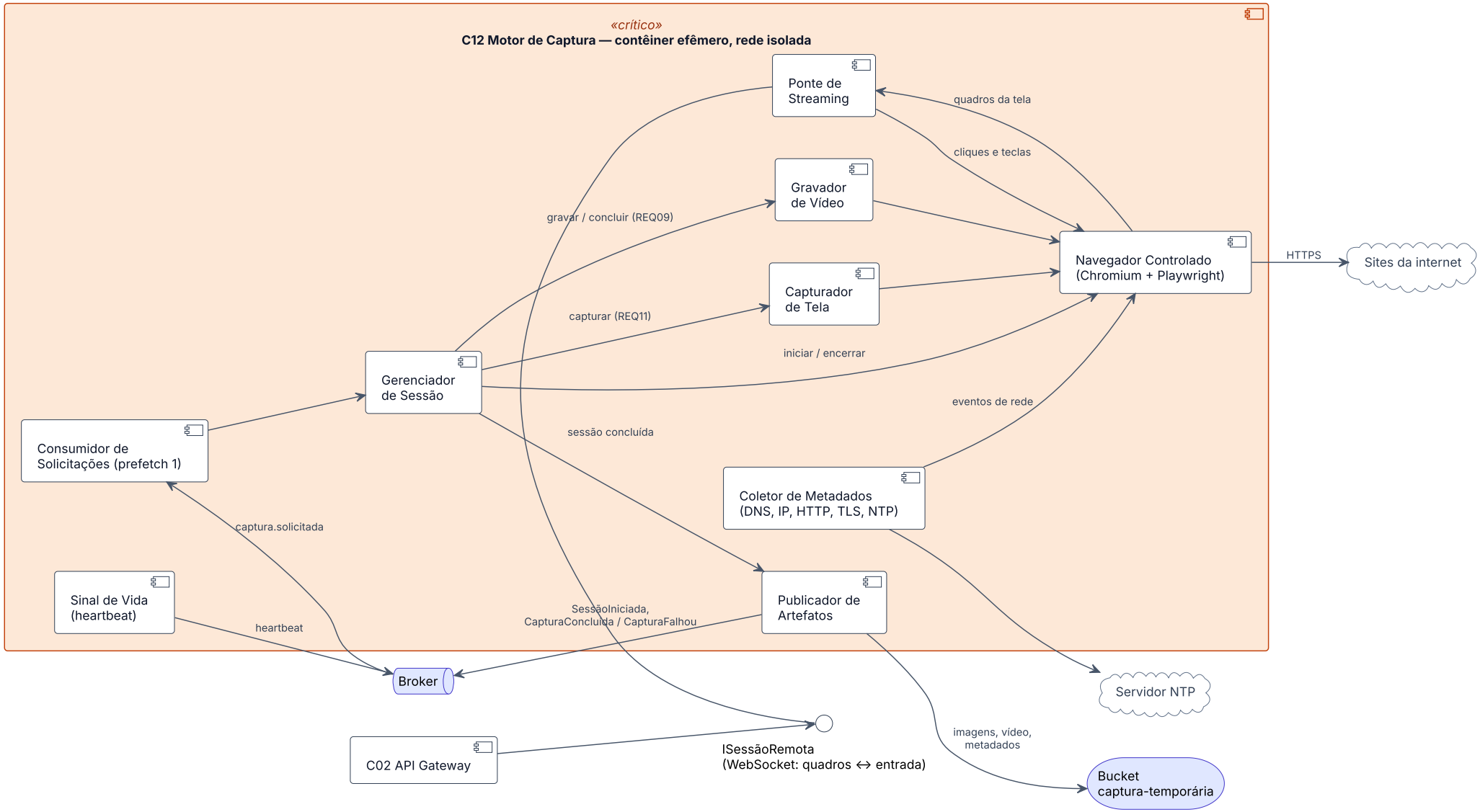
Subcomponente	Responsabilidade
Coordenador da Saga	Máquina de estados do registro (Iniciado → … → Concluído / Falhou / Cancelado)
Cliente da Carteira	Chama reservar, consumir e estornar com timeout e retentativa
Publicador de Solicitações	Publica `CapturaSolicitada` via outbox
Consumidor de Eventos	Recebe `SessãoIniciada`, `CapturaConcluída`, `CapturaFalhou`, `EvidênciaSelada`

Subcomponente	Responsabilidade
Alocador de Sessão	Registra no gateway a rota entre a sessão do usuário e o worker
Monitor de Prazos	Detecta sessões que não iniciaram, excederam a duração ou travaram
Repositório do Estado da Saga	Persiste o estado para retomar após falhas

Decisões relacionadas: ADR-0007, ADR-0008, ADR-0009.

7.5 C12 Motor de Captura

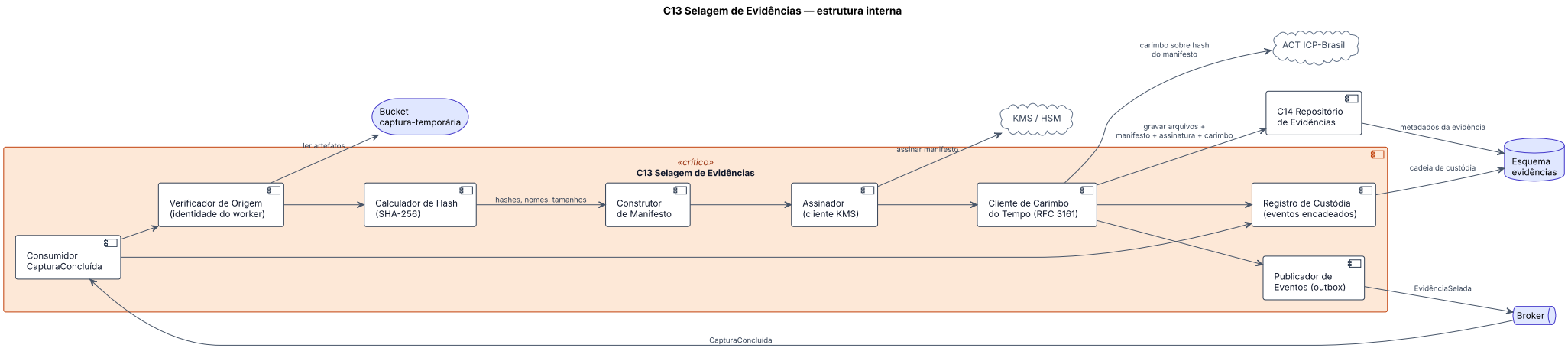
C12 Motor de Captura (worker) — estrutura interna



Subcomponente	Responsabilidade
Consumidor de Solicitações	Retira uma solicitação por vez da fila (prefetch 1)
Gerenciador de Sessão	Ciclo de vida do navegador, limites de duração, conclusão
Navegador Controlado	Chromium automatizado (ex.: Playwright) em contêiner efêmero e rede isolada
Ponte de Streaming	Envia quadros da tela e recebe cliques e teclas via WebSocket
Capturador de Tela	Gera capturas de tela (REQ11)
Gravador de Vídeo	Grava a navegação (REQ09)
Coletor de Metadados	Registra URL final, IP, DNS, cabeçalhos HTTP, certificado TLS e horário NTP
Publicador de Artefatos	Envia arquivos ao bucket temporário e publica `CapturaConcluída` / `CapturaFalhou`
Sinal de Vida	Envia heartbeat para o orquestrador detectar workers travados

Decisões relacionadas: ADR-0003, ADR-0009.

7.6 C13 Selagem de Evidências



Subcomponente	Responsabilidade
Consumidor `CapturaConcluída`	Recebe o aviso de captura concluída
Verificador de Origem	Confere que os artefatos vieram de um worker autenticado
Calculador de Hash	Calcula o SHA-256 de cada arquivo
Construtor de Manifesto	Monta o JSON com registro, usuário, URL, horários, metadados e hashes
Assinador	Assina o manifesto com a chave da plataforma no KMS
Cliente de Carimbo do Tempo	Obtém o carimbo RFC 3161 da ACT ICP-Brasil e grava o pacote no repositório WORM
Registro de Custódia	Registra cada etapa em eventos encadeados
Publicador de Eventos	Publica `EvidênciaSelada` via outbox

Decisão relacionada: ADR-0004.

8. Diagrama de módulos (pacotes UML)

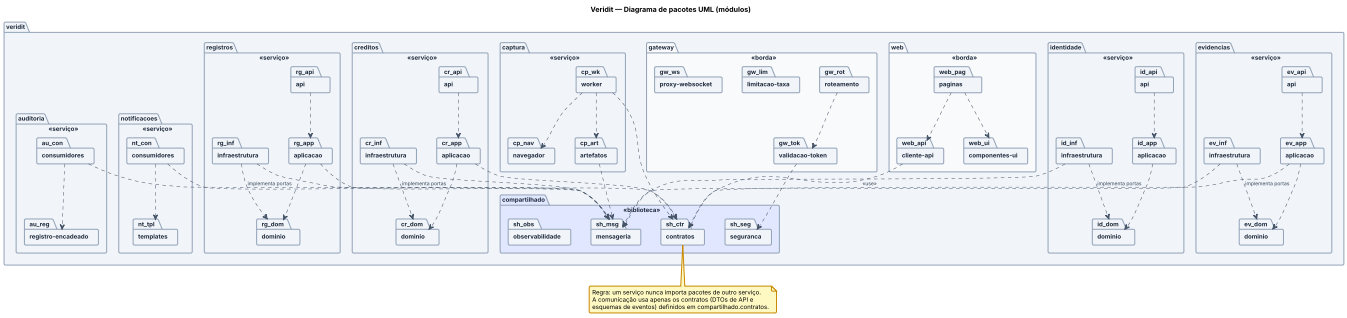


Diagrama de pacotes

Regras de dependência:

- **Entre serviços:** nenhum serviço importa pacotes de outro. A comunicação usa apenas os contratos em `compartilhado.contratos` (DTOs de API e esquemas de eventos).
- **Dentro dos serviços de domínio** (`identidade`, `creditos`, `registros`, `evidencias`): `api` → `aplicacao` → `dominio`. A `infraestrutura` implementa as portas definidas em `dominio` (inversão de dependência), então o domínio não depende de banco, broker ou provedores externos.
- **Biblioteca compartilhada:** `contratos`, `mensageria` (outbox, idempotência, cliente do broker), `seguranca` (validação de token) e `observabilidade` (logs e correlation-id). Não pode conter regra de negócio.

Pacote	Componentes
`web`	C01
`gateway`	C02
`identidade`	C03, C04, C05
`creditos`	C06, C07, C08, C09
`registros`	C10, C11
`captura`	C12
`evidencias`	C13, C14, C15, C16
`notificacoes`	C17
`auditoria`	C18

9. Táticas por característica

9.1 Segurança

Tática	Objetivo principal	Onde
Autenticar atores	Garantir que só usuários cadastrados acessem o sistema	C04, C02
Autorizar atores	Garantir que cada usuário acesse apenas os próprios registros e compras	C02, todos os serviços
Limitar acesso	Concentrar a entrada em um único ponto com TLS e limitação de taxa	C02
Limitar exposição	Expor só o gateway; manter serviços e captura em redes internas isoladas	C02, C12
Separar entidades	Isolar o navegador em contêiner efêmero e cada serviço em seu esquema de banco	C12, ADR-0010
Criptografar dados	Proteger dados em trânsito (TLS) e em repouso (AES-256, criptografia de campo)	Todos
Detectar ataques de força bruta	Identificar tentativas repetidas de login e recuperação e bloquear	C04
Revogar acesso	Encerrar sessões no logout, na troca de senha e em reuso de refresh token	C04
Minimizar dados sensíveis	Não receber nem armazenar dados de cartão	C08
Manter trilha de auditoria	Permitir investigar incidentes	C18

9.2 Integridade e autenticidade das evidências

Tática	Objetivo principal	Onde
Ambiente de captura controlado	Garantir que o conteúdo veio do site, sem manipulação no dispositivo do usuário	C12 (ADR-0003)
Verificar integridade (hash SHA-256)	Detectar qualquer alteração em qualquer arquivo	C13
Assinar o manifesto	Provar que o pacote foi emitido pela plataforma	C13 + KMS
Carimbo do tempo de terceiro confiável	Provar o momento da captura	C13 + ACT
Armazenamento imutável (WORM)	Impedir alteração ou remoção, inclusive por administradores	C14
Cadeia de custódia encadeada	Documentar cada etapa entre a captura e o pacote	C13, C18
Coletar metadados técnicos	Dar contexto à prova (IP, DNS, TLS, horário)	C12
Verificação periódica	Detectar corrupção silenciosa	C13 (job)

9.3 Confiabilidade

Tática	Objetivo principal	Onde
Outbox transacional	Não perder eventos entre a gravação no banco e a publicação	C08, C11, C13
Consumidores idempotentes	Obter efeito único mesmo com entrega repetida	C08, C09, C11, C17
Retentativa com backoff exponencial	Superar falhas transitórias sem sobrecarregar	C11, C13, C17
Fila de mensagens mortas (DLQ)	Isolar mensagens com erro sem travar a fila	Broker
Saga com compensação	Manter créditos coerentes quando a captura falha	C11, C09

Tática	Objetivo principal	Onde
Timeout e heartbeat	Detectar sessões e workers travados	C11, C12
Reconciliação	Corrigir webhooks perdidos	C08
Verificação de invariantes	Detectar inconsistência no saldo	C09

9.4 Escalabilidade / elasticidade

Tática	Objetivo principal	Onde
Escalonamento horizontal de workers sem estado	Aumentar a capacidade adicionando instâncias	C12
Autoescalonamento pela fila	Ajustar a capacidade à demanda (elasticidade)	C12 + orquestrador
Fila como buffer (nivelamento de carga)	Absorver picos sem rejeitar pedidos	Broker
Pool aquecido	Reduzir o tempo para iniciar novas sessões	C12
Limitar demanda	Controlar consumo (sessões por usuário, duração máxima)	C11, C02
Validação de token sem estado	Evitar gargalo no serviço de identidade	C02, C04
Cache de leituras	Reduzir carga em consultas frequentes (catálogo)	C06, C02

9.5 Disponibilidade

Tática	Objetivo principal	Onde
Redundância ativa	Continuar atendendo quando uma instância falha	Todos os serviços
Health check e reinício automático	Detectar e substituir instâncias com falha	Orquestrador de contêineres
Isolamento de falhas por serviço	Impedir que falha na captura derrube login e compras	Estilo (ADR-0002)
Circuit breaker	Evitar falhas em cascata quando um provedor externo cai	C08, C13, C17
Degradação graciosa	Manter listagem e downloads quando a captura estiver indisponível	C10, C16

9.6 Desempenho

Tática	Objetivo principal	Onde
Processamento assíncrono	Não bloquear a interface com selagem, relatório e .zip	C13, C15, C16
Paginação e índices	Responder rápido à listagem de registros	C10
Compressão adaptativa de quadros	Manter a interação com o navegador remoto fluida	C12
URLs pré-assinadas para download	Entregar arquivos sem passar pelos serviços	C14, C16

9.7 Recuperabilidade

Tática	Objetivo principal	Onde
Backup com recuperação ponto a ponto	Restaurar o banco após falha ou erro humano	PostgreSQL
Réplica em espera (failover)	Retomar o serviço rapidamente	PostgreSQL
Replicação do armazenamento de objetos	Não perder evidências em falha de zona	Armazenamento de objetos

Tática	Objetivo principal	Onde
Teste periódico de restauração	Garantir que o backup funciona	Operação

9.8 Auditabilidade

Tática	Objetivo principal	Onde
Log de auditoria somente de inserção com encadeamento de hashes	Registro de eventos que não pode ser alterado sem deixar rastro	C18
Correlation-id	Seguir uma operação entre serviços	`compartilhado.observabilidade`
Livro-razão de créditos	Histórico financeiro completo	C09

9.9 Privacidade (LGPD)

Tática	Objetivo principal	Onde
Minimização de dados	Coletar só os campos do Apêndice A; nunca dados de cartão	C03, C07, C08
Criptografia em nível de campo	Proteger CPF, telefone e endereço mesmo com acesso ao banco	C03, C07
Mascaramento em logs	Evitar exposição acidental de dados pessoais	`compartilhado.observabilidade`
Política de retenção	Manter dados apenas pelo prazo necessário	C14, PostgreSQL
Sessão de captura descartável	Não reter cookies ou credenciais digitadas pelo usuário	C12

9.10 Extensibilidade / configurabilidade

Tática	Objetivo principal	Onde
Parametrização (pacotes como dados)	Criar ou alterar pacotes sem mudar código (D2)	C06
Polimorfismo para tipos de usuário	Adicionar empresas (D1) sem alterar fluxos existentes	C03
Estratégia para tipos de captura	Acrescentar novos tipos de registro	C12
Publicar/assinar eventos	Adicionar consumidores sem alterar produtores	Broker
Portas e adaptadores	Trocar provedor de pagamento ou de carimbo do tempo	C08, C13

9.11 Interoperabilidade

Tática	Objetivo principal	Onde
Camada anticorrupção (adaptadores)	Isolar o modelo dos sistemas externos	C08, C13
Contratos versionados (OpenAPI e esquemas de eventos)	Manter a integração estável entre serviços	`compartilhado.contratos`
Padrões abertos (RFC 3161, SHA-256, X.509)	Permitir verificação por terceiros	C13

9.12 Manutenibilidade e testabilidade

Tática	Objetivo principal	Onde
Aumentar coesão	Concentrar cada domínio em um serviço	Estilo
Reduzir acoplamento	Proibir imports e joins entre serviços	Pacotes, ADR-0010

Tática	Objetivo principal	Onde
Inversão de dependência	Testar o domínio sem banco ou broker	Todos os serviços
Funções de aptidão no CI	Verificar regras arquiteturais automaticamente	Pipeline
Ambiente local completo (Docker Compose)	Reproduzir o sistema inteiro para testes	Operação

10. Decisões arquiteturais (ADRs)

As ADRs seguem o modelo padronizado em adr/0000-modelo-adr.md, com contexto, decisão, alternativas consideradas, justificativa, consequências, conformidade e requisitos relacionados.

ADR	Decisão	Características
0001	Top 4 características	Top 4
0002	Estilo baseado em serviços com mensageria assíncrona	Top 4
0003	Captura em navegador remoto isolado	Integridade, Segurança
0004	Selagem com hash, assinatura, carimbo do tempo e WORM	Integridade
0005	JWT curto + refresh token revogável; Argon2id	Segurança
0006	Pagamentos delegados ao Mercado Pago	Segurança, Confiabilidade
0007	Outbox, filas duráveis, DLQ e saga	Confiabilidade
0008	Carteira como livro-razão com reserva	Confiabilidade
0009	Workers de captura com autoescalonamento	Escalabilidade
0010	PostgreSQL com esquema por serviço + objetos	Segurança, Manutenibilidade

11. Rastreabilidade: requisitos → componentes

Requisito	Componentes
REQ01 Cadastrar usuário	C01, C02, C03, C17
REQ02 Recuperar senha	C05, C04, C17
REQ03 Logar	C04, C02
REQ04 Sair	C04, C02
REQ05 Comprar créditos	C06, C07, C08, C09
REQ06 Efetuar pagamento	C08
REQ07 Confirmar pagamento por e-mail	C08, C17
REQ08 Iniciar registro	C10, C11, C09, C12
REQ09 Concluir gravação	C12, C11, C13
REQ10 Confirmar gravação por e-mail	C11, C17
REQ11 Concluir captura de tela	C12, C11, C13
REQ11-B Confirmar captura de tela por e-mail	C11, C17
REQ12 Listar registros	C10
REQ13 Detalhes do registro	C10, C14
REQ14 Relatório	C15, C13
REQ15 Arquivo .zip	C16, C14
QA Segurança	C02, C04, C18

12. Riscos e pontos em aberto

Risco / pendência	Impacto	Encaminhamento
Consumo de créditos por tipo de registro não definido (P3)	Preço e modelo da carteira	Validar com o cliente; manter configurável
Interpretação de "pagamento pela plataforma" (P2)	Integração de pagamento	Validar com o cliente
Custo de infraestrutura da captura remota	Viabilidade financeira	Limites por usuário, teto de workers, monitorar custo por registro
Sites que bloqueiam IPs de datacenter	Capturas incompletas	Pool de saída de rede; registrar no relatório
Retenção WORM x pedidos de exclusão (LGPD)	Conformidade legal	Validar base legal com assessoria jurídica
Complexidade operacional (broker, orquestrador) para a equipe	Prazo de implementação	Docker Compose no desenvolvimento; autoescalonamento só em produção

13. Telas da aplicação

Protótipo no Figma a ser produzido na próxima etapa.

Referências

- RICHARDS, Mark; FORD, Neal. *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly.
- BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. *Software Architecture in Practice*. Addison-Wesley.
- ALMEIDA, Eduardo. *Documento de Requisitos — Veridit*, versão 01.00, 2026.
- IETF. RFC 3161 — Time-Stamp Protocol (TSP).
- BRASIL. Lei nº 13.709/2018 — Lei Geral de Proteção de Dados Pessoais.

Apêndice A — Registros de Decisão Arquitetural (ADRs)

As ADRs a seguir são as mesmas mantidas no repositório em `docs/arquitetura/adr/`. A primeira é o modelo padronizado usado por todas.

ADR-NNNN:

Status: Proposta | Aceita | Rejeitada | Substituída por ADR-XXXX | Depreciada
Data: AAAA-MM-DD
Decisores:
Características arquiteturais relacionadas:
Requisitos e restrições relacionados:

Contexto

Qual problema ou força motivou a decisão. Descreva a situação, as restrições e o que acontece se nada for decidido.

Decisão

A decisão em voz ativa e afirmativa ("Vamos adotar...", "O sistema usará...").

Alternativas consideradas

Alternativa	Prós	Contras
A	...	...
B	...	...

Justificativa

Por que a decisão foi escolhida em vez das alternativas. Relacione com as características arquiteturais e os requisitos. O "porquê" é mais importante que o "como".

Consequências

Positivas

- ...

Negativas / trade-offs

- ...

Riscos

- ...

Conformidade

Como a equipe vai verificar que a decisão está sendo seguida (funções de aptidão automatizadas, testes, revisões, métricas).

Notas

- Histórico de alterações do registro (o que mudou e por quê).
- ADRs relacionadas.
- Registro do uso de ferramentas de IA, quando houver (onde está o log da interação).

ADR-0001: Priorizar segurança, integridade das evidências, confiabilidade e escalabilidade

Status: Aceita

Data: 2026-10-15

Decisores:

Características arquiteturais relacionadas: Segurança, Integridade/Autenticidade das evidências, Confiabilidade, Escalabilidade (elasticidade da captura)

Requisitos e restrições relacionados: Atributo de qualidade "Segurança"; REQ01–REQ04 (acesso); REQ05–REQ07 (créditos e pagamento); REQ08–REQ11 (captura); REQ14–REQ15 (relatório e .zip); D1 (advogados como público)

Contexto

O Veridit registra conteúdo da web (captura de tela ou vídeo de navegação) e entrega um relatório e um pacote .zip com os arquivos capturados. Um dos públicos previstos são advogados (D1), o que indica que o produto final tende a ser usado como prova. O documento de requisitos explicita apenas uma característica (segurança); as demais são implícitas no domínio.

Levantamos 12 características candidatas (seção 2 do documento de arquitetura). Seguindo Richards & Ford, cada característica suportada aumenta a complexidade do sistema, então a arquitetura deve priorizar poucas: as que mais influenciam a estrutura e cuja falha inviabiliza o produto. As demais continuam atendidas, mas por táticas locais e não como direcionadoras do estilo.

Decisão

As quatro características prioritárias do Veridit, em ordem, são:

1. **Segurança** (explícita). Apenas usuários cadastrados acessam o sistema, cada usuário só acessa os próprios registros, e dados pessoais e de pagamento ficam protegidos. *Cenário mensurável:* 100% dos endpoints, exceto cadastro, login e recuperação de senha, rejeitam requisições sem token válido; nenhuma senha é armazenada em formato reversível.
2. **Integridade e autenticidade das evidências** (implícita). Deve ser possível demonstrar que os arquivos não foram alterados desde a captura e quando ela ocorreu. *Cenário:* a alteração de qualquer byte de um arquivo do pacote é detectada pela verificação do manifesto; 100% dos registros concluídos têm hash e carimbo do tempo.
3. **Confiabilidade** (implícita). Nenhum pagamento confirmado deixa de virar crédito, nenhum crédito é consumido sem registro concluído, e nenhuma captura concluída se perde. *Cenário:* com webhooks duplicados ou queda de um serviço no meio do fluxo, o efeito final é aplicado exatamente uma vez (0 créditos duplicados ou perdidos).
4. **Escalabilidade, com elasticidade no serviço de captura** (implícita). A captura é a operação mais cara (navegador dedicado e codificação de vídeo por minutos) e a demanda é irregular. *Cenário:* um aumento de 10x nas sessões simultâneas é absorvido apenas adicionando instâncias de captura, sem mudança de código, com tempo de espera p95 por uma sessão de até 30 s.

Os valores dos cenários são metas iniciais da equipe e devem ser validados com o cliente.

Alternativas consideradas

Alternativa	Prós	Contras
Disponibilidade no lugar de escalabilidade	Importante para qualquer SaaS	Não há SLA nos requisitos; uma indisponibilidade curta é tolerável se nada se perde (coberto por confiabilidade), e a redundância já vem do escalonamento horizontal
Desempenho no top 4	Melhora a experiência	O tempo da captura depende do site de destino; não diferencia o produto nem direciona o estilo
Auditabilidade como característica separada	Visível para o cliente	Já é coberta pela cadeia de custódia (integridade) e pelos logs de acesso (segurança)
Extensibilidade/configurabilidade no top 4 (D1, D2)	Antecipa empresas e novos pacotes	Atendida por parametrização e polimorfismo nos componentes, sem exigir um estilo específico
Privacidade (LGPD) como característica separada	Destaca dados pessoais (CPF, endereço)	Tratada dentro de segurança, com táticas específicas de proteção de dados

Justificativa

Escolhemos as características que, se falharem, tornam o produto inútil ou perigoso, e que mudam a forma da arquitetura. Segurança é a única explícita e envolve dados pessoais e de pagamento. Uma evidência que pode ser contestada por falta de integridade não tem valor para um advogado, então integridade é o núcleo do valor do produto. O usuário paga antes de usar (créditos), o que torna inaceitável perder pagamento ou crédito: daí a confiabilidade. Por fim, a captura tem perfil de carga diferente de todo o resto (pesada, longa e irregular), o que exige escalar só essa parte. Essas quatro forças, juntas, levam a separar a captura do núcleo transacional e a usar comunicação assíncrona (ADR-0002).

Consequências

Positivas

- Critério claro para escolher o estilo (ADR-0002) e para resolver trade-offs futuros.
- Cada característica prioritária tem um cenário mensurável e pelo menos uma função de aptidão.

Negativas / trade-offs

- Custo maior que o de uma aplicação simples: carimbo do tempo, armazenamento imutável, broker de mensagens e orquestração de contêineres.
- Desempenho e disponibilidade recebem menos investimento inicial.

Riscos

- Se o cliente definir um SLA de disponibilidade rígido, esta ADR deve ser revista.

Conformidade

- Revisar a lista sempre que uma nova versão do documento de requisitos for publicada.
- Cada característica prioritária tem funções de aptidão listadas na seção 9 do documento de arquitetura (táticas) e nas ADRs 0003 a 0010.

Notas

- ADRs relacionadas: 0002 (estilo), 0003 e 0004 (integridade), 0005 e 0006 (segurança), 0006, 0007 e 0008 (confiabilidade), 0009 (escalabilidade).
- Elaborada em sala com apoio de ferramenta de IA; registro da interação em . Revisada e aceita pela equipe.

ADR-0002: Adotar arquitetura baseada em serviços com mensageria assíncrona

Status: Aceita

Data: 2026-10-15

Decisores:

Características arquiteturais relacionadas: Segurança, Integridade, Confiabilidade, Escalabilidade (top 4, ADR-0001); Manutenibilidade; Custo

Requisitos e restrições relacionados: REQ01–REQ15; restrições do projeto: equipe pequena, prazo acadêmico, etapas futuras de implementação e evolução

Contexto

O domínio se divide naturalmente em áreas com pouca dependência entre si: identidade e acesso, créditos e pagamentos, gestão de registros, captura, evidências (selagem, relatório e .zip) e notificações. Essas áreas têm perfis de carga diferentes. Cadastro, login e listagens são operações curtas de CRUD. A captura dura de segundos a minutos, consome muita CPU e memória e executa conteúdo externo não confiável. Os e-mails (REQ07, REQ10, REQ11-B) são assíncronos por natureza.

A equipe é pequena e o projeto terá etapas de implementação e evolução, então o estilo precisa ser simples o bastante para ser construído e testado pela equipe.

Decisão

Vamos adotar o estilo **baseado em serviços** (*service-based*), particionado por domínio:

- Seis serviços de domínio implantados separadamente: Identidade e Acesso, Créditos e Pagamentos, Registros, Captura, Evidências e Notificações, mais um serviço transversal de Auditoria.
- Uma aplicação web (SPA) acessa os serviços por um **API Gateway**.
- Comunicação **síncrona (REST)** para comandos e consultas da interface, e **assíncrona (eventos em broker de mensagens)** para captura, selagem, notificações e auditoria (ADR-0007).
- Um banco de dados compartilhado, com esquema isolado por serviço (ADR-0010).
- O serviço de Captura não acessa o banco e se comunica só por fila e armazenamento de objetos, formando um quantum arquitetural próprio, escalável de forma independente (ADR-0009).

Alternativas consideradas

Classificação por estrelas de Richards & Ford (1 = fraco, 5 = forte):

Característica	Camadas	Microkernel	Baseado em serviços	Orientado a eventos	Space-based	Microsserviços
Escalabilidade	1	1	3	5	5	5
Elasticidade	1	1	2	3	5	5
Tolerância a falhas	1	1	4	5	3	4
Confiabilidade	3	3	4	3	4	4
Testabilidade	2	3	4	2	1	4
Implantabilidade	1	3	4	3	3	4
Simplicidade	5	4	3	1	1	1

Característica	Camadas	Microkernel	Baseado em serviços	Orientado a eventos	Space-based	Microsserviços
Custo (5 = barato)	5	5	4	3	2	1

Alternativa	Prós	Contras
Monólito em camadas	Mais simples e barato	Escalabilidade, elasticidade e tolerância a falhas = 1: um pico de capturas derrubaria login e pagamentos; o navegador que abre sites externos ficaria no mesmo processo que os dados sensíveis
Microkernel	Bom para variações por plug-in	Não resolve escalabilidade nem isolamento; o núcleo do Veridit não é um produto de plug-ins
Microsserviços	Escalabilidade e elasticidade máximas	Custo e simplicidade = 1; exige banco por serviço e transações distribuídas para créditos; desproporcional para 15 requisitos e uma equipe pequena
Orientado a eventos puro	Escalabilidade e tolerância a falhas = 5	Testabilidade = 2, simplicidade = 1; consultas da interface (listar, detalhar) ficam mais difíceis; tratamento de erros complexo
Space-based	Elasticidade máxima para picos	Feito para alto volume concorrente sobre dados em memória, que não é o caso; testabilidade = 1 e alto custo

Justificativa

O estilo baseado em serviços tem boas notas justamente nas características prioritárias que dependem da estrutura: tolerância a falhas e confiabilidade (4), além de testabilidade, implantabilidade e custo (4) adequados à equipe. Seus pontos fracos, elasticidade (2) e escalabilidade (3), aparecem quando todo o sistema precisa escalar. No Veridit, só a captura precisa disso, então tratamos esse ponto separando a Captura em um serviço com workers sem estado alimentados por fila (ADR-0009). Usamos eventos só onde a assincronia é natural, sem pagar o custo de um sistema totalmente orientado a eventos.

Para segurança e integridade, a separação em serviços também isola o componente mais arriscado (o navegador que carrega sites externos) dos dados pessoais e de pagamento, e concentra a selagem das evidências em um único serviço auditável.

Consequências

Positivas

- Falha ou sobrecarga na captura não afeta login, compras e downloads.
- Cada serviço pode ser implantado e testado separadamente.
- Caminho de evolução: um serviço pode ser dividido em microsserviços no futuro, se necessário.

Negativas / trade-offs

- Custos de sistema distribuído: latência de rede, consistência eventual entre Registros e Evidências, necessidade de observabilidade (correlation-id) e de um broker.
- O banco compartilhado acopla os serviços que o usam (mitigado pelo isolamento por esquema).

Riscos

- Proliferação de chamadas síncronas entre serviços. Mitigação: chamadas síncronas só para a reserva de créditos; o restante por eventos.

Conformidade

- Função de aptidão no CI: nenhum serviço importa pacotes internos de outro serviço; só o pacote `compartilhado/contratos` (regras de dependência, ex.: dependency-cruiser ou ArchUnit).
- Cada serviço usa um usuário de banco com permissão apenas no próprio esquema.

- A imagem do serviço de Captura não contém driver nem credenciais de banco.

Notas

- Referência: Richards, M.; Ford, N. *Fundamentals of Software Architecture*, capítulos sobre estilos (camadas, microkernel, baseado em serviços, orientado a eventos, space-based e microsserviços).
- ADRs relacionadas: 0001, 0007, 0009, 0010.
- Elaborada em sala com apoio de ferramenta de IA; registro da interação em .

ADR-0003: Realizar a captura em navegador remoto isolado, executado no servidor

Status: Aceita

Data: 2026-10-15

Decisores:

Características arquiteturais relacionadas: Integridade e autenticidade das evidências, Segurança, Escalabilidade

Requisitos e restrições relacionados: REQ08 (iniciar registro: tela ou vídeo de navegação), REQ09 (concluir gravação), REQ11 (concluir captura de tela), REQ14 (relatório com URL e site)

Contexto

O usuário informa o endereço do site e o título do registro e pode capturar uma tela ou gravar um vídeo da navegação (REQ08). Se a captura acontecer no dispositivo do usuário (extensão de navegador, print local ou aplicativo), o próprio usuário pode alterar a página antes de capturar (DevTools, edição do HTML, arquivo hosts, imagem editada). Nesse caso a plataforma não consegue afirmar que o conteúdo estava de fato publicado no site.

Além disso, abrir sites arbitrários é arriscado: a página pode conter código malicioso.

Decisão

A captura será feita **no servidor**, por workers do serviço de Captura:

- Cada sessão roda em um **contêiner efêmero** com navegador Chromium controlado por automação (ex.: Playwright).
- A tela do navegador remoto é transmitida para a aplicação web, e os cliques e teclas do usuário são repassados ao navegador por WebSocket. O usuário navega, mas o conteúdo é renderizado e capturado no ambiente da plataforma.
- Capturas de tela e gravação de vídeo são produzidas pelo worker, junto com **metadados técnicos**: URL final, IP do servidor de destino, resolução DNS, cabeçalhos HTTP, certificado TLS do site e horário sincronizado por NTP.
- O contêiner é destruído ao final, sem cookies ou cache persistentes, e roda em rede isolada, sem acesso à rede interna da plataforma.
- Limites por sessão: duração máxima (ex.: 30 min) e número de sessões simultâneas por usuário.

Alternativas consideradas

Alternativa	Prós	Contras
Extensão de navegador no computador do usuário	Barata; sem custo de servidor; navegação fluida	Ambiente não confiável: o usuário controla o DOM e a rede; evidência facilmente contestável
Aplicativo desktop de captura	Mais controle que a extensão	Continua sendo ambiente do usuário; custo de distribuição e atualização
Captura automática só pela URL, sem interação	Simples; fácil de escalar	Não atende "vídeo de navegação"; não alcança conteúdo que exige rolagem, cliques ou login
API de screenshot de terceiros	Rápida de integrar	Cadeia de custódia fora do controle da plataforma; dependência externa; não grava navegação

Justificativa

Capturar em ambiente controlado pela plataforma é o que permite afirmar como, quando e de onde o conteúdo foi obtido, que é a base da integridade e da autenticidade (ADR-0001). O isolamento em contêiner efêmero atende à segurança, porque o conteúdo externo nunca é executado perto dos dados sensíveis. O modelo de workers sem estado é o que permite escalar a captura de forma independente (ADR-0009).

Consequências

Positivas

- Evidência com maior força probatória; ambiente padronizado e reproduzível.
- Metadados técnicos coletados automaticamente para o relatório (REQ14).

Negativas / trade-offs

- Custo de infraestrutura por sessão (CPU, memória e banda).
- Latência na interação perceptível para o usuário.
- Complexidade do streaming da tela.

Riscos

- Sites que bloqueiam IPs de datacenter. Mitigação: pool de saída de rede com IPs identificáveis e documentados no relatório.
- O usuário pode digitar credenciais pessoais (ex.: rede social) dentro da sessão remota. Mitigação: aviso na interface; campos de senha não aparecem no vídeo (são mascarados pelo próprio navegador); a sessão é destruída ao final.

Conformidade

- Teste automatizado de política de rede: o contêiner de captura não alcança endereços da rede interna.
- O serviço de Evidências só aceita artefatos publicados por workers autenticados (identidade do worker via mTLS).

Notas

- ADRs relacionadas: 0001, 0004, 0009.
- Elaborada em sala com apoio de ferramenta de IA; registro da interação em .

ADR-0004: Selar as evidências com hash, manifesto assinado, carimbo do tempo e armazenamento imutável

Status: Aceita

Data: 2026-10-15

Decisores:

Características arquiteturais relacionadas: Integridade e autenticidade das evidências, Auditabilidade, Segurança

Requisitos e restrições relacionados: REQ09, REQ11 (conclusão da captura), REQ13 (detalhes), REQ14 (relatório com nome e tamanho dos arquivos), REQ15 (.zip); Apêndice A (campos do relatório)

Contexto

Para que o registro tenha valor como prova, é preciso demonstrar a terceiros (juiz, perito, parte contrária) que os arquivos não foram alterados desde a captura e em que momento a captura ocorreu. Guardar os arquivos e um hash no banco da própria plataforma não basta: quem administra o banco poderia alterar os dois.

Decisão

Ao fim de cada captura, o serviço de Evidências vai:

1. Calcular o **SHA-256** de cada arquivo capturado.
2. Montar um **manifesto** (JSON) com id do registro, usuário, URL, data/hora de início e fim, metadados técnicos e a lista de arquivos com nome, tamanho e hash.
3. **Assinar** o manifesto com a chave privada da plataforma, guardada em serviço de gerenciamento de chaves (KMS/HSM) e nunca exportada.
4. Obter um **carimbo do tempo** (RFC 3161) de uma Autoridade de Carimbo do Tempo credenciada na ICP-Brasil sobre o hash do manifesto.
5. Gravar arquivos, manifesto, assinatura e carimbo em **armazenamento de objetos com bloqueio de escrita (WORM / Object Lock)** pelo prazo de retenção.
6. Registrar cada passo na **cadeia de custódia** (trilha de auditoria com encadeamento de hashes).

O relatório (REQ14) inclui os hashes e instruções de verificação. O .zip (REQ15) contém os arquivos, o manifesto, a assinatura, o carimbo e o relatório, permitindo verificação offline.

O registro só muda para "Concluído" depois de selado.

Alternativas consideradas

Alternativa	Prós	Contras
Apenas hash guardado no banco	Simples e sem custo	Quem controla o banco altera arquivo e hash juntos; sem prova de data
Registro do hash em blockchain pública	Verificação independente e pública	Taxas e latência; menor familiaridade de juízes e peritos; mais complexidade; pode ser extensão futura
Assinatura digital sem carimbo do tempo	Prova a origem	Não prova quando a captura ocorreu
Ata notarial em cartório	Fé pública reconhecida	Lenta e cara; é justamente o processo que o produto busca simplificar

Justificativa

A combinação de hash, assinatura e carimbo do tempo de terceiro confiável permite que qualquer pessoa verifique, sem depender do Veridit, que o pacote não mudou e que existia na data indicada. O armazenamento imutável impede alteração ou remoção, inclusive por administradores. A trilha de custódia documenta o caminho do arquivo desde o worker até o pacote. Juntas, essas táticas atendem diretamente à característica de integridade (ADR-0001).

Consequências

Positivas

- Pacote verificável por perito, offline, com ferramentas comuns (sha256sum, OpenSSL).
- Qualquer adulteração é detectável.

Negativas / trade-offs

- Custo por carimbo do tempo e por armazenamento retido.
- Dependência da disponibilidade da Autoridade de Carimbo do Tempo. Mitigação: a selagem é assíncrona com novas tentativas; o registro fica em "Selando" até o carimbo ser obtido.
- Arquivos sob retenção WORM não podem ser apagados antes do prazo, mesmo a pedido do usuário. Base legal a validar: exercício regular de direitos em processo (LGPD, art. 7º, VI).

Riscos

- Comprometimento da chave da plataforma. Mitigação: chave em KMS/HSM, rotação e carimbo do tempo independente.

Conformidade

- Job diário que, por amostragem, recalcula hashes e valida assinatura e carimbo de registros concluídos; qualquer divergência gera alerta crítico.
- Teste automatizado: tentar sobrescrever ou apagar um objeto do bucket de evidências deve falhar.

Notas

- ADRs relacionadas: 0001, 0003, 0010.
- Elaborada em sala com apoio de ferramenta de IA; registro da interação em .

ADR-0005: Autenticar com token de acesso curto e refresh token revogável, com senhas em Argon2id

Status: Aceita

Data: 2026-10-15

Decisores:

Características arquiteturais relacionadas: Segurança, Escalabilidade

Requisitos e restrições relacionados: Atributo de qualidade "Segurança" (acesso só por usuários cadastrados); REQ01 (cadastro), REQ02 (recuperar senha), REQ03 (login), REQ04 (logout); D1 (advogado, usuário comum e, futuramente, empresa)

Contexto

Vários serviços precisam saber quem é o usuário em cada requisição. Consultar o serviço de Identidade a cada chamada cria acoplamento e um ponto único de falha. Ao mesmo tempo, o logout (REQ04) precisa ter efeito real e a recuperação de senha (REQ02) é um vetor clássico de ataque.

Decisão

- O serviço de Identidade e Acesso emite um **token de acesso JWT** assinado com chave assimétrica, válido por **15 minutos**, com id do usuário e perfil (advogado, comum).
- Emite também um **refresh token** opaco e **rotativo**, guardado com hash no banco, válido por 7 dias e enviado em cookie `HttpOnly`, `Secure`, `SameSite=Strict`. Reuso de um refresh token já trocado revoga toda a família de tokens (indício de roubo).
- O **API Gateway** valida a assinatura do JWT com as chaves públicas (JWKS) e repassa a identidade; cada serviço autoriza pelo dono do recurso e pelo perfil.
- **Logout (REQ04):** revoga o refresh token e coloca o `jti` do token de acesso em uma lista de revogação no cache do gateway até ele expirar.
- **Senhas:** hash com **Argon2id**; política mínima de senha; verificação de e-mail no cadastro.
- **Recuperação de senha (REQ02):** token aleatório de uso único (256 bits), guardado com hash, válido por 30 min, enviado por e-mail; a resposta não revela se o e-mail existe; após a troca, todas as sessões são revogadas.
- **Proteção contra força bruta:** limite de tentativas por IP e por conta, com bloqueio progressivo.

Alternativas consideradas

Alternativa	Prós	Contras
Sessão no servidor (cookie + armazenamento de sessão)	Revogação imediata; simples	Gateway ou serviços precisam consultar o armazenamento de sessão a cada requisição
JWT de longa duração sem refresh	Muito simples	Logout não tem efeito real; token roubado vale por muito tempo
Provedor de identidade externo (Keycloak, Auth0, Cognito)	Maduro; MFA e SSO prontos	Custo e operação adicionais; considerado para quando houver contas de empresa (D1)
bcrypt em vez de Argon2id	Amplamente suportado	Argon2id é a recomendação atual da OWASP, por resistir melhor a ataques com GPU

Justificativa

Tokens curtos permitem que o gateway valide a identidade localmente, sem chamada extra, o que escala com o número de serviços. O refresh token rotativo e a lista de revogação dão ao logout efeito imediato.

Argon2id, tokens de recuperação de uso único e limitação de tentativas cobrem os ataques mais comuns contra login e recuperação de senha.

Consequências

Positivas

- Validação de identidade sem estado nos serviços.
- Logout efetivo e detecção de roubo de refresh token.

Negativas / trade-offs

- Gestão e rotação de chaves de assinatura.
- A lista de revogação no gateway adiciona um pequeno estado (cache).

Riscos

- Vazamento da chave privada de assinatura. Mitigação: chave em KMS e rotação periódica.

Conformidade

- Função de aptidão no CI: varre a especificação OpenAPI e verifica que todo endpoint não público responde 401 sem token.
- Teste de autorização: acessar registro de outro usuário retorna 404.
- Análise estática (SAST) e de dependências (SCA) no pipeline.

Notas

- ADRs relacionadas: 0001, 0002.
- Elaborada em sala com apoio de ferramenta de IA; registro da interação em .

ADR-0006: Delegar pagamentos ao Mercado Pago (checkout e Pix), sem armazenar dados de pagamento

Status: Aceita

Data: 2026-10-15

Decisores:

Características arquiteturais relacionadas: Segurança, Confiabilidade, Interoperabilidade

Requisitos e restrições relacionados: REQ05 (comprar créditos), REQ06 (pagamento pelo Mercado Pago ou pela plataforma, com QR Code/código Pix), REQ07 (e-mail de confirmação); D2 (pacotes); Apêndice A (dados de faturamento)

Contexto

O REQ06 exige duas formas de pagamento: pelo Mercado Pago ou pela própria plataforma do Veridit, gerando QR Code ou código Pix. Processar cartões diretamente coloca o sistema no escopo completo do PCI DSS. Além disso, a confirmação de pagamento é assíncrona: um Pix pode ser pago minutos depois de gerado, e o aviso chega por webhook, que pode atrasar, se repetir ou se perder.

Decisão

O componente de Processamento de Pagamentos vai integrar com o Mercado Pago por meio de um **adaptador** (porta `GatewayPagamento`):

- **"Pagar com Mercado Pago":** cria uma preferência no Checkout do Mercado Pago e redireciona o usuário.
- **"Pagar pela plataforma Veridit":** cria um pagamento Pix pela API do provedor e exibe, na interface do Veridit, o QR Code e o código copia-e-cola, com prazo de expiração (ex.: 30 min).
- **Confirmação:** o webhook tem a assinatura validada; o sistema consulta o status na API do provedor (não confia no corpo do webhook) e processa de forma **idempotente** pelo id do pagamento. Um job de **reconciliação** periódica busca pagamentos pendentes para cobrir webhooks perdidos.
- Ao confirmar: o pedido muda para "Pago", a carteira recebe os créditos (ADR-0008) e o evento `PagamentoConfirmado` dispara o e-mail (REQ07).
- O Veridit guarda apenas os dados de faturamento do Apêndice A e os ids do provedor; **nunca dados de cartão**.

Alternativas consideradas

Alternativa	Prós	Contras
Processar cartões direto com uma adquirente	Taxas potencialmente menores	Escopo completo do PCI DSS; risco e custo de conformidade altos
Pix direto com banco (API Pix)	Sem intermediário para o Pix	Exige conta PJ, certificados mTLS e conciliação própria; viável em versão futura
Outro provedor (Stripe, PagSeguro, Asaas)	Alternativas maduras	O requisito cita o Mercado Pago; o adaptador permite trocar depois

Justificativa

Delegar a captura dos dados de pagamento ao provedor reduz drasticamente o escopo de segurança (o sistema nunca vê dados de cartão). Usar um único provedor para as duas opções do REQ06 simplifica a integração e a conciliação. Validação de webhook, consulta de status, idempotência e reconciliação

garantem que todo pagamento confirmado vire crédito exatamente uma vez, atendendo à confiabilidade (ADR-0001).

Consequências

Positivas

- Menor escopo de PCI DSS; nenhum dado de cartão armazenado.
- Troca de provedor isolada no adaptador.

Negativas / trade-offs

- Dependência de um fornecedor e de suas taxas.
- Indisponibilidade do provedor impede compras (mas não afeta capturas de quem já tem créditos).

Riscos

- Interpretação do REQ06: assumimos que "pela plataforma do Veridit" significa pagar via Pix sem sair da interface do Veridit, ainda que o Pix seja gerado pelo provedor. **Validar com o cliente.**

Conformidade

- Teste de idempotência: o mesmo webhook processado duas vezes gera um único lançamento de crédito.
- Verificação no CI de que nenhuma migração cria colunas de número de cartão ou CVV.
- Alerta para pedidos "Pendentes" com Pix expirado sem conciliação.

Notas

- ADRs relacionadas: 0001, 0007, 0008.
- Elaborada em sala com apoio de ferramenta de IA; registro da interação em .

ADR-0007: Comunicar de forma assíncrona com outbox transacional, filas duráveis e DLQ

Status: Aceita

Data: 2026-10-15

Decisores:

Características arquiteturais relacionadas: Confiabilidade, Escalabilidade, Disponibilidade

Requisitos e restrições relacionados: REQ05–REQ07 (compra → crédito → e-mail), REQ08–REQ11 (registro → captura → selagem → e-mail), REQ10, REQ11-B

Contexto

Os fluxos principais atravessam vários serviços: pagamento → créditos → e-mail, e registro → captura → selagem → conclusão → e-mail. Se um serviço grava no banco e cai antes de publicar o evento (problema da escrita dupla), o fluxo para no meio: crédito pago e não creditado, ou captura feita e registro nunca concluído. Chamadas síncronas em cadeia, por sua vez, tornam o fluxo frágil e não combinam com capturas que duram minutos.

Decisão

- Usar um **broker de mensagens** (RabbitMQ ou equivalente gerenciado) com filas duráveis e confirmação manual (*ack*).
- Produtores usam **outbox transacional**: o evento é gravado na mesma transação que a mudança de estado, e um *relay* publica e marca como enviado.
- Consumidores são **idempotentes** (tabela de mensagens já processadas por id do evento).
- Falhas têm **retentativas com backoff exponencial**; após N falhas, a mensagem vai para uma **fila de mensagens mortas (DLQ)** com alerta.
- Eventos levam `correlationId` e esquema versionado, definido no pacote `compartilhado/contratos`.
- O fluxo do registro é uma **saga orquestrada** pelo serviço de Registros: reservar créditos → solicitar captura → aguardar selagem → consumir créditos; em falha, estornar a reserva (compensação).

Alternativas consideradas

Alternativa	Prós	Contras
Só chamadas REST síncronas	Simples de entender e depurar	Acopla a disponibilidade dos serviços; timeouts em capturas longas; sem buffer para picos
Publicar direto no broker após o commit (sem outbox)	Menos código	Perde eventos se o serviço cair entre o commit e a publicação
Kafka	Alto volume e replay de eventos	Operação pesada; desproporcional ao volume do Veridit
Transação distribuída (2PC)	Consistência forte	Não suportada pela maioria dos brokers; bloqueios e baixa disponibilidade

Justificativa

O outbox garante que nenhum evento se perca; filas duráveis com ack garantem a entrega ao menos uma vez; consumidores idempotentes transformam isso em efeito exatamente uma vez. A saga com compensação garante que nenhum crédito fique preso ou seja perdido. A fila também serve de buffer para picos de captura (ADR-0009).

Consequências

Positivas

- Nenhum pagamento ou captura se perde por falha parcial.
- Serviços podem ficar fora do ar por algum tempo sem perder trabalho.

Negativas / trade-offs

- Consistência eventual: o usuário vê estados intermediários ("Processando", "Selando").
- Mais infraestrutura (broker) e complexidade de depuração.

Riscos

- Mensagens acumulando na DLQ sem tratamento. Mitigação: alerta quando a DLQ não estiver vazia e procedimento de reprocessamento documentado.

Conformidade

- Teste de caos: derrubar o consumidor no meio do processamento; a mensagem é reentregue e o efeito é aplicado uma única vez.
- Métricas com alerta: tamanho da DLQ > 0; atraso do outbox p95 > 5 s.

Notas

- ADRs relacionadas: 0002, 0006, 0008, 0009.
- Elaborada em sala com apoio de ferramenta de IA; registro da interação em .

ADR-0008: Modelar a carteira de créditos como livro-razão com reserva, consumo e estorno

Status: Aceita

Data: 2026-10-15

Decisores:

Características arquiteturais relacionadas: Confiabilidade, Integridade (dos dados financeiros), Auditabilidade

Requisitos e restrições relacionados: REQ05 (comprar créditos), REQ06 (pagamento), REQ08 (iniciar registro consome créditos); D2 (pacotes)

Contexto

Créditos comprados (REQ05) são consumidos ao registrar conteúdo (REQ08). Uma coluna de "saldo" atualizada diretamente tem três problemas: condições de corrida (duas capturas simultâneas com um único crédito), créditos perdidos quando a captura falha e falta de histórico para resolver contestações do usuário.

Premissa: a quantidade de créditos consumida por tipo de registro (tela ou vídeo, e por duração do vídeo) não está definida nos requisitos; será configurável no catálogo e deve ser validada com o cliente.

Decisão

- A carteira será um **livro-razão somente de inserção**, com lançamentos do tipo `CREDITO_COMPRA`, `RESERVA`, `CONSUMO`, `ESTORNO_RESERVA` e `AJUSTE`.
- O saldo disponível é materializado com **controle de concorrência** (versão otimista ou bloqueio por carteira) e sempre igual à soma dos lançamentos.
- Ao iniciar um registro, o serviço de Registros **reserva** os créditos (chamada síncrona); sem saldo, a operação é recusada e o usuário é levado à compra.
- Quando o registro é selado, a reserva vira **consumo**; em falha, cancelamento ou timeout, a reserva é **estornada**. Reservas têm prazo de validade para não ficarem presas.
- Toda operação tem **chave de idempotência** (id do registro ou do pagamento).

Alternativas consideradas

Alternativa	Prós	Contras
Coluna de saldo simples	Muito simples	Corridas, sem histórico, créditos perdidos em falhas
Debitar só no fim da captura	Nunca cobra captura que falhou	Usuário sem saldo iniciaria capturas caras; risco de saldo negativo
Debitar no início, sem reserva	Simples	Exige fluxo de reembolso em toda falha; menos explícito que a reserva

Justificativa

O livro-razão torna cada movimentação rastreável e permite recalcular o saldo a qualquer momento. A reserva com consumo ou estorno garante que o usuário nunca perde crédito por falha da plataforma e que não há gasto duplo, atendendo à confiabilidade (ADR-0001).

Consequências

Positivas

- Histórico completo para suporte e contestação.
- Sem gasto duplo e sem créditos perdidos.

Negativas / trade-offs

- Modelo e consultas mais complexos que uma coluna de saldo.
- Necessidade de job de expiração de reservas e de verificação de invariantes.

Conformidade

- Job de verificação: saldo materializado = soma dos lançamentos, para todas as carteiras.
- Teste de concorrência: N reservas simultâneas com saldo para apenas uma resultam em exatamente um sucesso.

Notas

- ADRs relacionadas: 0006, 0007.
- Elaborada em sala com apoio de ferramenta de IA; registro da interação em .

ADR-0009: Escalar a captura com pool de workers sem estado e autoescalonamento pela fila

Status: Aceita
Data: 2026-10-15
Decisores:
Características arquiteturais relacionadas: Escalabilidade (elasticidade), Disponibilidade, Custo
Requisitos e restrições relacionados: REQ08–REQ11 (captura de tela e vídeo)

Contexto

Cada sessão de captura precisa de um navegador dedicado (ordem de 1 vCPU e 1–2 GB de memória, mais na codificação de vídeo) durante minutos. A demanda tende a ser irregular, com picos em horário comercial e perto de prazos processuais. Os demais serviços são leves. Dimensionar tudo para o pico desperdiça recursos; dimensionar para a média gera filas longas no pico.

Decisão

- O serviço de Captura roda como **workers sem estado** em contêineres, gerenciados por um orquestrador (ex.: Kubernetes; em desenvolvimento, Docker Compose com réplicas fixas).
- Pedidos de sessão entram na fila `captura.solicitada`. Cada worker atende **uma sessão por vez** e só busca a próxima quando fica livre (*prefetch* = 1).
- Um **autoescalador** (ex.: KEDA) ajusta o número de workers pela profundidade da fila e pelo número de sessões ativas, entre um mínimo (**pool aquecido**, ex.: 2) e um máximo (teto de custo).
- **Limites de demanda:** até 2 sessões simultâneas por usuário e duração máxima por sessão.
- Os artefatos vão para uma área temporária no armazenamento de objetos; o worker **não acessa o banco de dados**.

Alternativas consideradas

Alternativa	Prós	Contras
Escalar o sistema inteiro junto (monólito)	Uma única unidade para operar	Desperdício: replica partes leves para atender à captura
Funções serverless (ex.: AWS Lambda)	Escala a zero; paga por uso	Limites de tempo e memória; streaming interativo difícil; cold start
Pool fixo dimensionado para o pico	Previsível	Custo ocioso alto fora do pico
Pool fixo pequeno	Barato	Filas longas no pico; experiência ruim

Justificativa

Workers sem estado alimentados por fila são a forma mais direta de obter elasticidade só onde ela é necessária, compensando a elasticidade limitada do estilo baseado em serviços (ADR-0002). A fila funciona como buffer de carga, e o pool aquecido reduz o tempo de espera pela primeira sessão. Como o worker não acessa o banco, ele forma um quantum arquitetural independente, com características operacionais próprias.

Consequências

Positivas

- Custo proporcional à demanda.

- Falha de um worker afeta só uma sessão; o orquestrador reinicia o contêiner.

Negativas / trade-offs

- Exige infraestrutura de orquestração de contêineres e conhecimento da equipe para operá-la.
- Partida de novos nós leva tempo (mitigado pelo pool aquecido).

Riscos

- Custos fora de controle em um pico anômalo. Mitigação: teto de workers e limites por usuário.

Conformidade

- Teste de carga: com 10x sessões, tempo de espera p95 por sessão ≤ 30 s após o escalonamento.
- Métrica com alerta: tempo de espera na fila `captura.solicitada`.
- Função de aptidão: a imagem do worker não contém credenciais nem driver de banco.

Notas

- ADRs relacionadas: 0002, 0003, 0007.
- Elaborada em sala com apoio de ferramenta de IA; registro da interação em .

ADR-0010: Usar PostgreSQL compartilhado com esquemas isolados por serviço e armazenamento de objetos para arquivos

Status: Aceita

Data: 2026-10-15

Decisores:

Características arquiteturais relacionadas: Segurança (privacidade), Manutenibilidade, Recuperabilidade, Integridade

Requisitos e restrições relacionados: Apêndice A (dados de usuários, compras e registros); REQ12–REQ15 (consultas, relatório e .zip)

Contexto

No estilo baseado em serviços é comum compartilhar o banco de dados, o que simplifica a operação. Porém tabelas compartilhadas criam acoplamento: uma mudança de esquema pode quebrar vários serviços, e qualquer serviço consegue ler dados que não deveria (ex.: faturamento). O sistema também lida com arquivos grandes (imagens, vídeos, relatórios, .zip), que não devem ficar dentro do banco relacional.

Decisão

- Uma instância **PostgreSQL** (com réplica e backups com recuperação ponto a ponto), com um **esquema por serviço**: `identidade`, `creditos`, `registros`, `evidencias`, `auditoria`.
- Cada serviço conecta com **usuário de banco próprio**, com permissão apenas no próprio esquema. Não há *joins* entre esquemas; dados de outro serviço vêm por API ou evento.
- Arquivos em **armazenamento de objetos compatível com S3**, em buckets separados: `captura-temporaria`, `evidencias` (com Object Lock, ADR-0004) e `relatorios`.
- Criptografia em repouso em ambos; CPF, telefone e endereço também **criptografados em nível de campo** pela aplicação.

Alternativas consideradas

Alternativa	Prós	Contras
Banco por serviço (estilo microsserviços)	Independência total	Mais custo operacional; consultas e consistência entre serviços ficam mais difíceis
Um esquema único compartilhado	Simples	Alto acoplamento; qualquer serviço lê dados de pagamento e pessoais
Arquivos dentro do banco (BLOB)	Uma única fonte	Banco grande, backups pesados, sem bloqueio WORM

Justificativa

O isolamento por esquema e usuário mantém a simplicidade operacional do banco compartilhado e limita o acoplamento e a exposição de dados, como recomendam Richards & Ford para o particionamento lógico do banco no estilo baseado em serviços. Separar os arquivos em armazenamento de objetos permite o bloqueio WORM exigido pela integridade e mantém o banco leve.

Consequências

Positivas

- Operação simples, com transações locais em cada serviço.

- Caminho de evolução para banco por serviço: os esquemas já estão separados.

Negativas / trade-offs

- O banco é ponto único de falha e capacidade compartilhada (mitigado por réplica e failover).
- Os serviços que usam o banco formam um único quantum arquitetural; aceitável porque só a Captura precisa de elasticidade independente.

Conformidade

- Verificação no CI: as migrações do serviço X só alteram o esquema X.
- Auditoria periódica das permissões (*grants*) do banco.
- Teste de restauração de backup a cada mês.

Notas

- ADRs relacionadas: 0002, 0004.
- Elaborada em sala com apoio de ferramenta de IA; registro da interação em .